

Atlas 200 DK V100R020C00 User Guide

Issue 01
Date 2021-04-07



Copyright © Huawei Technologies Co., Ltd. 2021. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Contents

1 Introduction.....	1
2 Preparing Accessories and a Development Server.....	2
3 Hardware Installation.....	4
3.1 Removing the Upper Case.....	4
3.2 Installing a Camera (PCB IT21DMDA).....	5
3.3 Installing a Camera (PCB IT21VDMB).....	10
4 Software Installation.....	15
4.1 Installing the Operating Environment.....	15
4.1.1 Creating an SD Card.....	15
4.1.1.1 Overview.....	15
4.1.1.2 Creating an SD Card with a Card Reader.....	15
4.1.1.3 Creating an SD Card Without a Card Reader.....	20
4.1.2 Connecting the Atlas 200 DK to the Ubuntu Server.....	26
4.2 Installing the Development Environment.....	30
4.2.1 Obtaining Software Packages.....	30
4.2.2 Configuring Ubuntu x86.....	31
4.2.3 Installing the Development Kit.....	35
4.2.4 Deploying the Media Module.....	37
4.2.5 Configuring the Cross-Compilation Environment.....	38
4.2.6 Installing MindStudio.....	39
4.3 Installing the Development Environment and Operating Environment.....	39
5 Common Operations.....	50
5.1 Upgrade and Uninstallation.....	50
5.1.1 Upgrading Atlas 200 DK.....	50
5.1.2 Upgrading the Development Kit.....	55
5.1.3 Uninstalling the Development Kit.....	56
5.2 Powering on Atlas 200 DK.....	57
5.3 Powering off the Atlas 200 DK Developer Board.....	61
5.4 Connecting the Atlas 200 DK over a Serial Port.....	61
5.5 Checking the Software Versions of Atlas 200 DK.....	63
5.6 Checking the Version of the Motherboard of the Developer Board.....	64
5.7 Checking the Version of the Atlas 200 AI Accelerator Module.....	67

5.8 Checking the Firmware Version of the Developer Board.....	71
5.9 Viewing the Channel to Which a Camera Belongs.....	72
5.10 Installing the Windows USB Network Adapter Driver.....	72
5.11 Setting User Account Validity Period.....	76
5.12 Installing CMake 3.5.2.....	77
5.13 Configuring a System Network Proxy.....	78
5.14 Parameters.....	78
6 FAQs.....	81
6.1 What Do I Do If the "Software Has Been Installed" During RUN Package Installation?.....	81
6.2 The error message "subprocess.CalledProcessError: Command '('lsb_release', '-a')' return non-zero exit status 1 " is displayed during pip3.7.5 installation.....	82
6.3 What Do I Do If "Could not find a version that satisfies the requirement xxx" Is Displayed When pip3.7.5 install Is Run?.....	83
6.4 What Do I Do If a Redundant Mounted Disk Appears Due to Manual Removal of the SD Card During SD Card Creation?.....	84
6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?.....	84
6.6 What Do I Do If the Atlas 200 DK Cannot Connect to the Ubuntu Server?.....	85

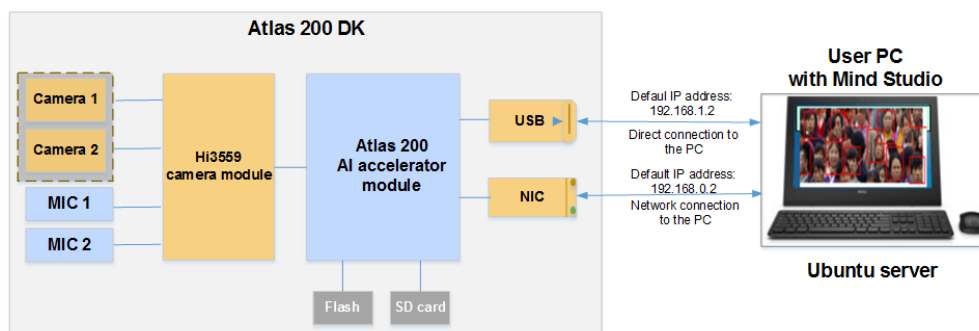
1 Introduction

Huawei Atlas 200 developer kit (DK) is a developer board product powered by the Huawei Ascend 310 chipset for one-stop AI app development.

This document describes the preparations for using the Atlas 200 DK to develop and run AI applications, including creating an SD card, connecting the Atlas 200 DK to the Ubuntu server, and installing the development tool.

The following figure shows the system block diagram of the Atlas 200 DK:

Figure 1-1 Connection between the Atlas 200 DK developer board and Mind Studio



NOTE

In the preceding figure, 192.168.1.2/192.168.0.2 is the IP address of the Atlas 200 DK, which can be selected during card making.

The Atlas 200 DK contains the Hi3559 camera module and Atlas 200 AI accelerator module. The PC where MindStudio is located is connected to the Atlas 200 DK through the USB port or network cable.

MindStudio contains the development kit and tool modules (such as the model management tool, compilation tool, and log tool). The development kit provides the library files, tools, dependencies, and common header files required for device compilation.

2

Preparing Accessories and a Development Server

This section describes how to prepare the accessories and the development server for using the Atlas 200 DK.

Preparing Accessories

Table 2-1 lists the accessories needed to be purchased in advance for using the Atlas 200 DK.

Table 2-1 Accessories

Name	Description	Recommended Model
Secure Digital (SD) card	Creates the startup system for the Atlas 200 DK developer board.	Tested and recommended SD cards: • Samsung UHS-I U3 CLASS 10 64 GB • Kingston UHS-I U1 CLASS 10 64 GB
Camera	Provides video streams for the Atlas 200 DK. For details, see 3.2 Installing a Camera (PCB IT21DMDA) and 3.3 Installing a Camera (PCB IT21VDMB) .	The Atlas 200 DK is compatible with Raspberry Pi cameras (yellow flat cables are required).
(Optional) Camera support	Fixes the camera. For details, see 3.2 Installing a Camera (PCB IT21DMDA) and 3.3 Installing a Camera (PCB IT21VDMB) .	Raspberry Pi transparent camera support
Type-C cable	Used by the Ubuntu server to connect to the Atlas 200 DK using a USB cable.	USB 3.0 Type-C cable

Name	Description	Recommended Model
Network cable	Used by the Ubuntu server to connect to the Atlas 200 DK using a network cable	Common network cable with RJ45 connectors
(Optional) Serial cable	Used for viewing the startup logs when the Atlas 200 DK startup indicator is abnormal, or the SD card is successfully prepared but the UI Host cannot be accessed. For details, see 6.6 What Do I Do If the Atlas 200 DK Cannot Connect to the Ubuntu Server? .	USB-to-TTL serial cable with 3.3 V interface level

Preparing a Server

Prepare a Ubuntu server featuring the x86 architecture for the following scenarios:

- Create a bootable SD card for the Atlas 200 DK. The card reader or Atlas 200 DK connects to the Ubuntu server through the USB port to create the system boot disk for the Atlas 200 DK.
- The server can be used to deploy the development environment and develop applications.
- Ubuntu OS version: 18.04.4 or 18.04.5
Download the installation package of the corresponding version from <http://releases.ubuntu.com/releases/> and install it.
You can download **ubuntu-*{version}*-desktop-amd64.iso** (desktop edition) or **ubuntu-*{version}*-server-amd64.iso** (server edition). *{version}* indicates the OS version.
- Python 3.x must exist in the Ubuntu OS.
- The free space of the system is greater than 20 GB.
- The system memory is greater than 4 GB.

3 Hardware Installation

3.1 Removing the Upper Case

3.2 Installing a Camera (PCB IT21DMDA)

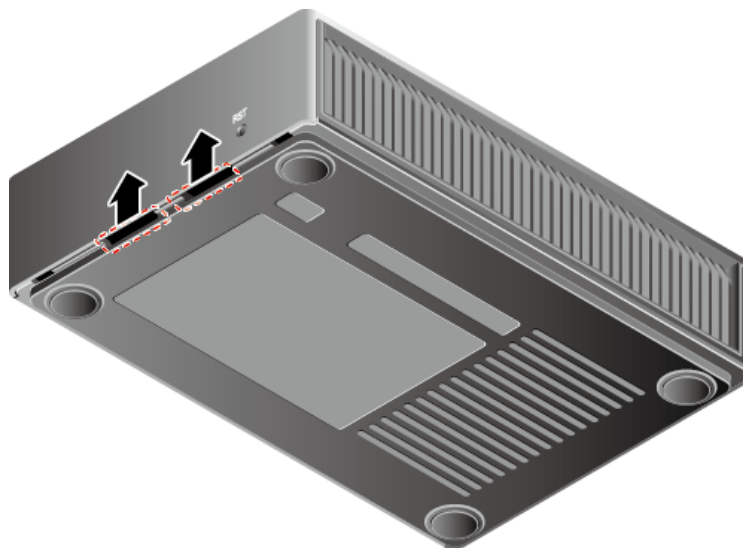
3.3 Installing a Camera (PCB IT21VDMB)

3.1 Removing the Upper Case

To use a camera or an internal port, remove the top cover from the Atlas 200 DK as follows:

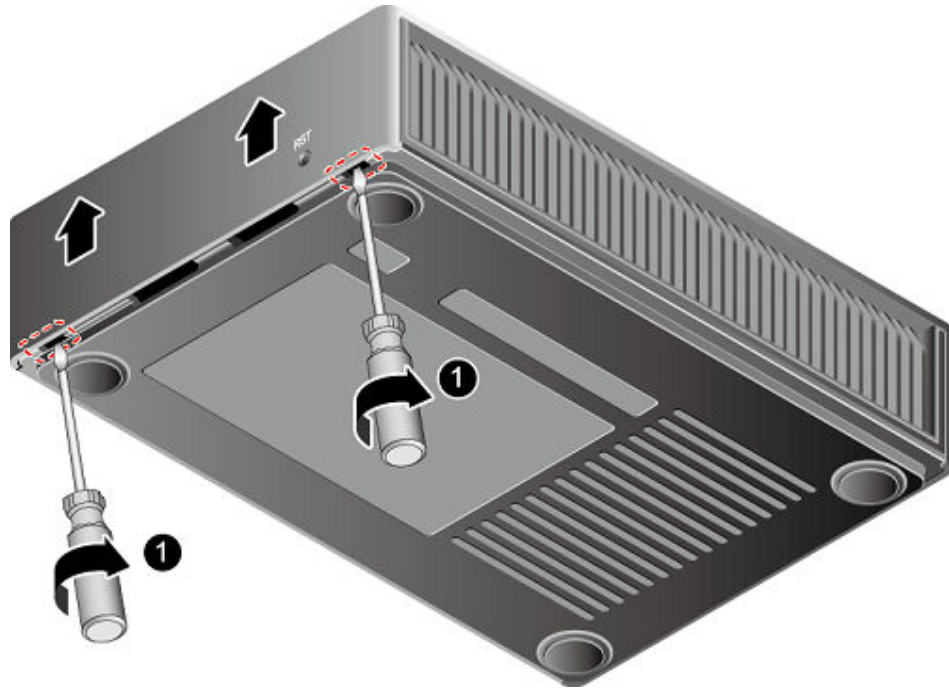
- Step 1** Check whether a camera cable is lead out from the Atlas 200 DK developer board.
- If yes, go to [Step 3](#).
 - If not, go to [Step 2](#).
- Step 2** If no camera cable is lead out from the Atlas 200 DK developer board, pull the plastic latch upwards to loosen the upper case, as shown in [Figure 3-1](#).

Figure 3-1 Removing the upper case - 1



- Step 3** If a camera cable is lead out from the Atlas 200 DK developer board, insert a flat-head screwdriver into the groove between the upper case and the bottom plate, and rotate the screwdriver to pry off the upper case, as shown in (1) in [Figure 3-2](#).

Figure 3-2 Removing the upper case



- Step 4** Remove the upper case.

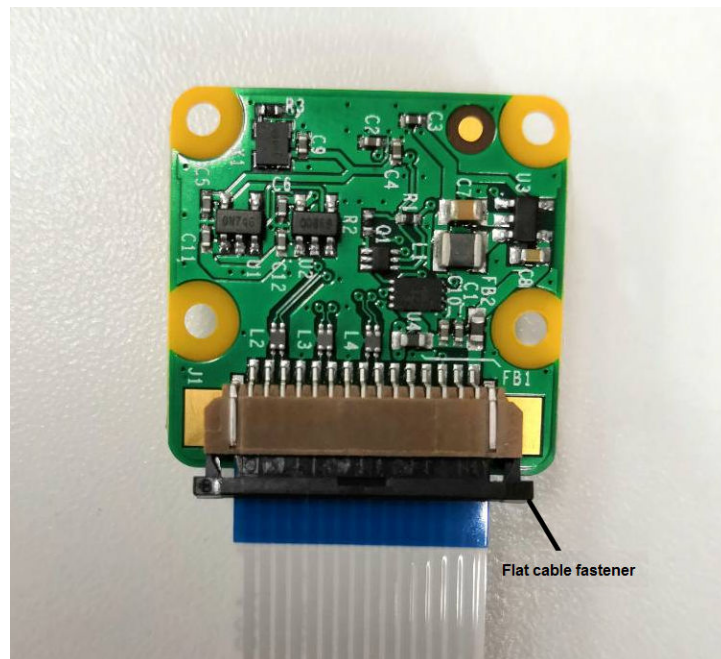
----End

3.2 Installing a Camera (PCB IT21DMDA)

Procedure

- Step 1** Replace the white flat cable delivered with the Raspberry Pi camera with a yellow camera flat cable.
1. Remove the black flat cable fastener from the camera. See [Figure 3-3](#).

Figure 3-3 Flat cable fastener



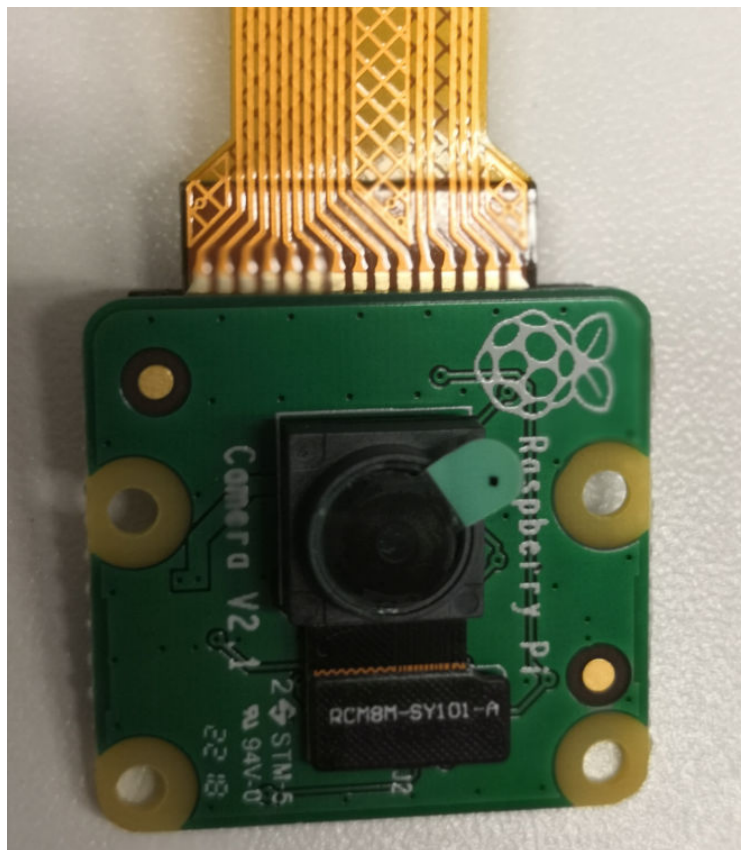
2. Take out the white camera flat cable. See [Figure 3-4](#).

Figure 3-4 White camera flat cable



3. Place the metal wire at the wide end of the yellow camera flat cable upwards and horizontally insert it into the cable trough of the camera until the cable cannot move. See [Figure 3-5](#).

Figure 3-5 Connecting the camera



4. Secure the black flat cable fastener.

Step 2 Install the fixing film on the camera head to the yellow camera flat cable. See [Figure 3-6](#).

Figure 3-6 Installing the fixing film



Step 3 Connect the camera flat cable to the Atlas 200 DK developer board.

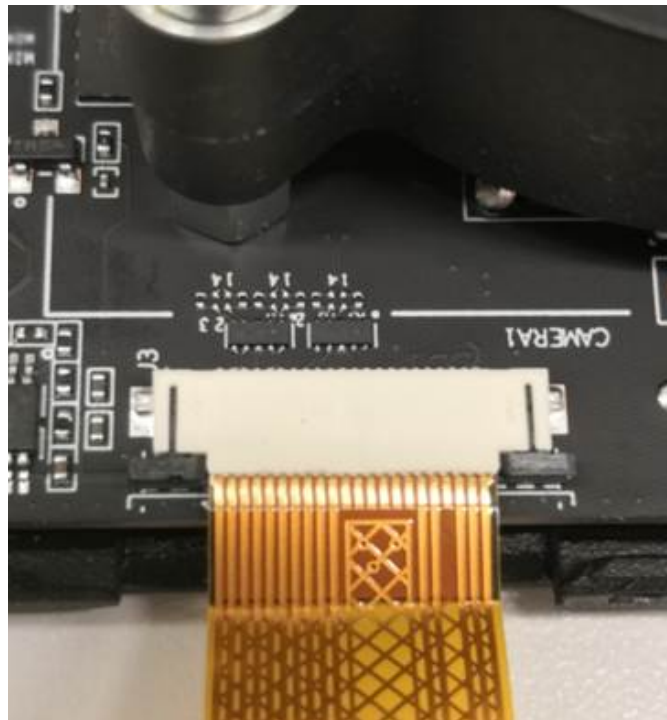
1. Remove the camera connector fastener from the Atlas 200 DK developer board. See [Figure 3-7](#).

Figure 3-7 Removing the black fastener



2. Place the metal wire at the narrow end of the yellow camera flat cable upwards and horizontally insert it into the camera connector CAMERA0 or CAMERA1 on the Atlas 200 DK developer board until the cable cannot be moved. Insert the fastener. See [Figure 3-8](#).

Figure 3-8 Inserting the fastener

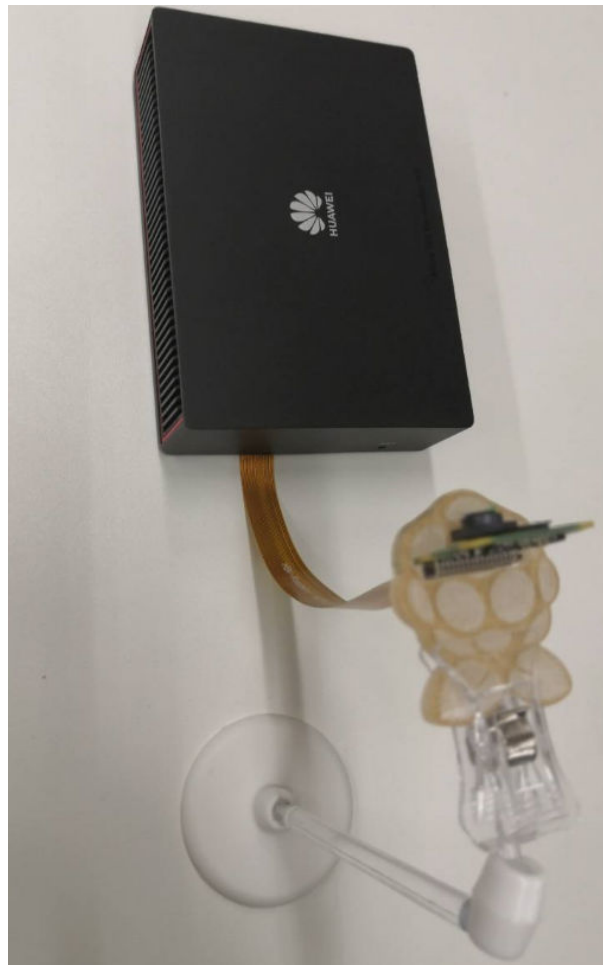


Step 4 Install the upper cover of the Atlas 200 DK developer board to the original position.

Step 5 Install the camera support.

1. Use the clip on the camera support to clamp the fixing film. See [Figure 3-9](#).

Figure 3-9 Installing the camera support



2. Install the camera support and support the camera, as shown in the preceding figure.

NOTE

- Before using the camera, remove the protective film from the camera.
- The base of the camera support has double-sided tape, which can be used to secure the support on the desktop to ensure that the camera is securely installed.

----End

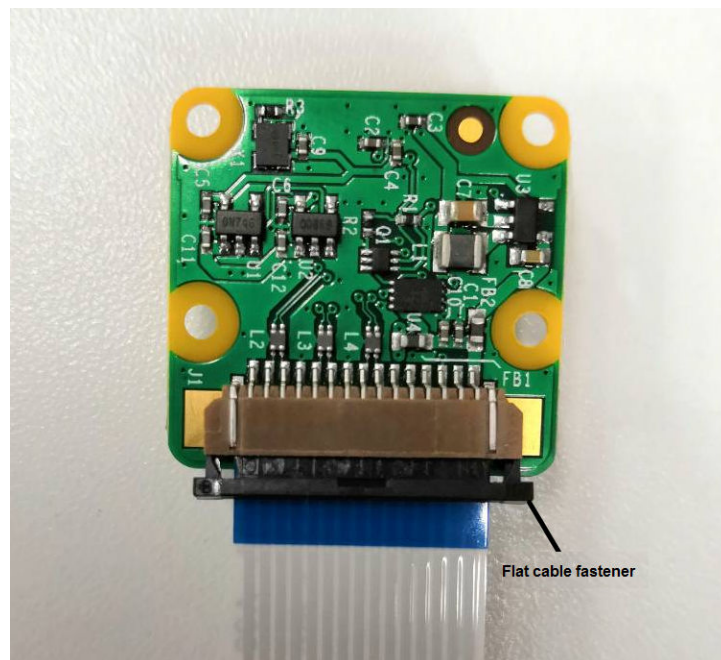
3.3 Installing a Camera (PCB IT21VDMB)

Procedure

- Step 1** Replace the white flat cable delivered with the Raspberry Pi camera with a black camera flat cable.

1. Remove the black flat cable fastener from the camera. See [Figure 3-10](#).

Figure 3-10 Removing the flat cable fastener



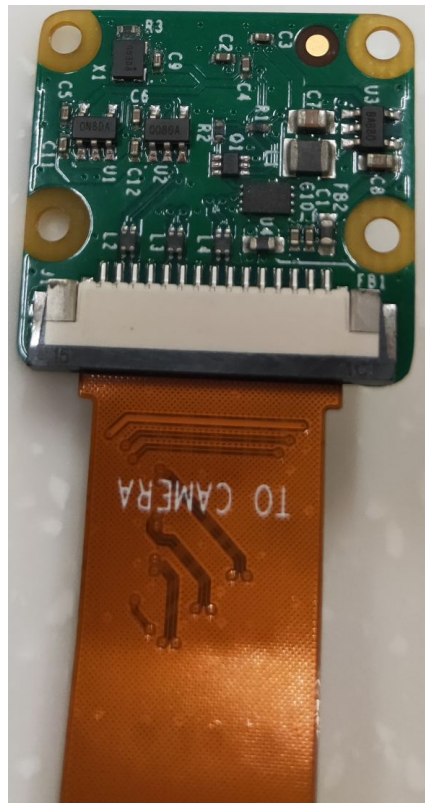
2. Take out the white camera flat cable. See [Figure 3-11](#).

Figure 3-11 White camera flat cable



3. Place the metal wire at the end with the silkscreen **TO CAMERA** of the black camera flat cable upwards and horizontally insert it into the cable trough of the camera until the cable cannot move. See [Figure 3-12](#).

Figure 3-12 Connecting the camera



4. Secure the black flat cable fastener.

Step 2 Install the fixing film on the camera head to the black camera flat cable.

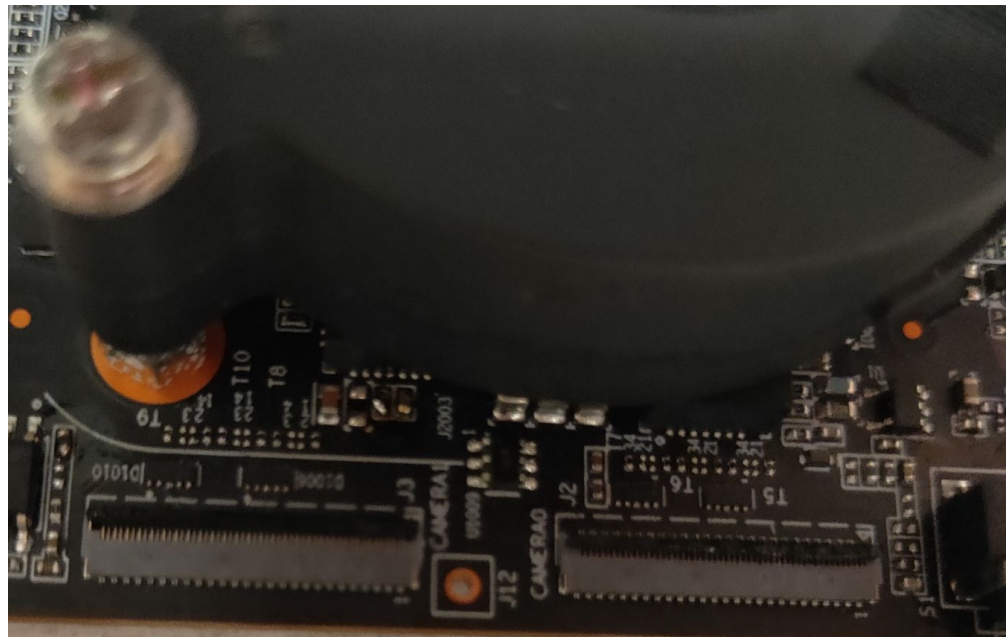
Step 3 Connect the camera flat cable to the Atlas 200 DK developer board.

NOTICE

- The connector fastener can be opened at a maximum of 90 degrees. When opening the connector fastener upwards, do not exceed 90 degrees.
- Do not open the connector fastener in the reverse direction. Otherwise, the connector will break.

-
1. Open the camera connector fastener of the Atlas 200 DK developer board upwards at 90 degrees. See [Figure 3-13](#).

Figure 3-13 Opening the black fastener



2. Place the black wire at the end with the silkscreen **TO MAIN BD** of the black camera flat cable upwards and horizontally insert it into the camera connector CAMERA0 or CAMERA1 on the Atlas 200 DK developer board until the cable cannot be moved. Insert the fastener. See [Figure 3-14](#).

Figure 3-14 Inserting the fastener

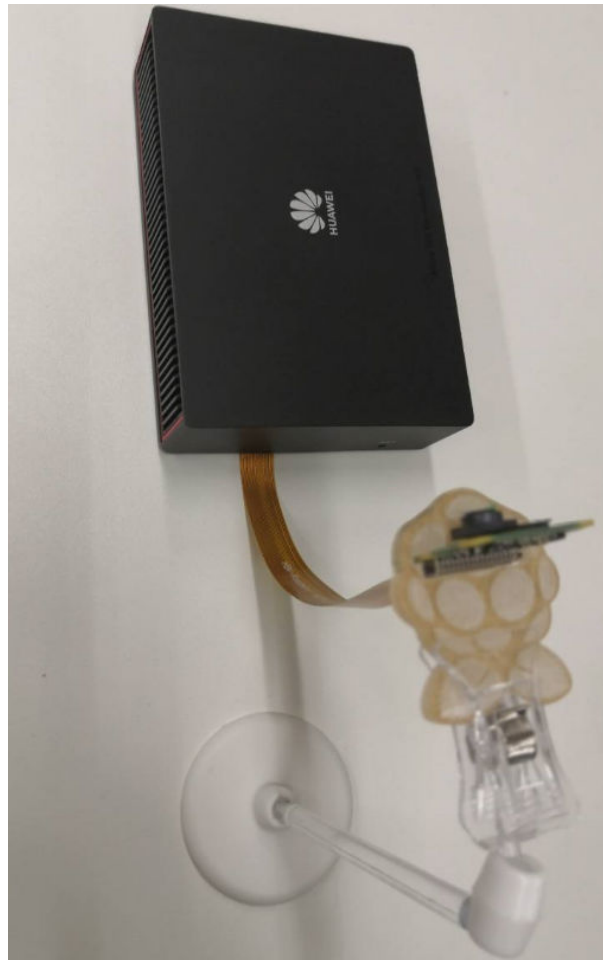


Step 4 Install the upper cover of the Atlas 200 DK developer board to the original position.

Step 5 Install the camera support.

1. Use the clip on the camera support to clamp the fixing film. See [Figure 3-15](#).

Figure 3-15 Installing the camera support



2. Install the camera support and support the camera, as shown in the preceding figure.

NOTE

- Before using the camera, remove the protective film from the camera.
- The base of the camera support has double-sided tape, which can be used to secure the support on the desktop to ensure that the camera is securely installed.

----End

4 Software Installation

[4.1 Installing the Operating Environment](#)

[4.2 Installing the Development Environment](#)

[4.3 Installing the Development Environment and Operating Environment](#)

4.1 Installing the Operating Environment

4.1.1 Creating an SD Card

4.1.1.1 Overview

You can create a system boot disk for the Atlas 200 DK by preparing an SD card.

You can use either of the following methods to prepare an SD card:

- If a card reader is available, insert the SD card into the card reader, connect the card reader to the USB port of the Ubuntu server, and run the SD card preparation script.
- If no card reader is available, insert the SD card into the card slot of the Atlas 200 DK, use a jumper cap/wire to connect the pins of the Atlas 200 DK, connect the Atlas 200 DK to the USB port of the Ubuntu server, and run the SD card preparation script.

NOTICE

During SD card preparation, the default user **HwHiAiUser** is automatically created for running the applications.

The default login password of the **HwHiAiUser** user is **Mind@123**.

4.1.1.2 Creating an SD Card with a Card Reader

This section describes how to connect a card reader to the Ubuntu server over the USB port and run SD card preparation scripts with a card reader.

Hardware Preparation

Prepare a 16-GB or larger SD card.

NOTE

The SD card will be formatted. Back up your data in advance.

Software Preparation

Table 4-1 describes how to obtain the Driver packages and runfile of Atlas 200 DK and the Ubuntu OS image.

Table 4-1 Required files

Descripti on	File Name	Details	URL
Ubuntu OS image	ubuntu-18.04.xx-server-arm64.iso	OS image of the Atlas 200 DK. Ubuntu OS version: 18.04.4 or 18.04.5. Must be a server release for ARM hardware.	Link
Driver packages and runfile of the Atlas 200 DK	Ascend310-driver- <i>{software version}</i> -ubuntu18.04.aarch64-minirc.tar.gz	Driver package, including OS peripheral software, AI software stack, maintenance and testing software, as well as drivers. During SD card preparation, Firmware information is obtained from the Driver package, so the Firmware component is not needed.	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.
	Ascend310-aicpu_kernels- <i>{software version}</i> -minirc.tar.gz	AI CPU OPP.	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.

Descri ption	File Name	Details	URL
	Ascend-acclib- <i>{software version}</i> - ubuntu18.04.aarch6 4-minirc.run	Ascend Computing Language Library (ACCLib) runfile, which is used to build and run AscendCL applications, enabling model loading and execution. During SD card creation, install the runfile as the HwHiAiUser user. The LD_LIBRARY_PATH environment variable will be automatically set in the .bashrc file of the HwHiAiUser user. export LD_LIBRARY_PATH=/ home/HwHiAiUser/ Ascend/acclib/lib64	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.

NOTICE

Keep the names of the downloaded files unchanged.

Procedure

- Step 1** Insert the SD card into the card reader and then insert the card reader into the USB port of the Ubuntu server.
- Step 2** Run the following commands on the Ubuntu server to install the qemu-user-static, binfmt-support, YAML, and cross compiler:

su - root

Update the sources:

apt-get update

Install the Python dependencies:

pip3 install pyyaml

apt-get install qemu-user-static binfmt-support python3-yaml gcc-aarch64-linux-gnu g++-aarch64-linux-gnu

Step 3 Run the following command as the **root** user on the Ubuntu server to create a card creation project directory:

```
mkdir /home/ascend/mksd
```

The card creation project directory is user-defined.

Step 4 Upload the obtained Ubuntu OS image package and Driver packages of the Atlas 200 DK to the card creation project directory (for example, **/home/ascend/mksd**).

Step 5 Run the following commands in the card creation project directory (for example, **/home/ascend/mksd**) to download the card creation scripts:

- Download the **make_sd_card.py** script:

```
wget https://raw.githubusercontent.com/Ascend/tools/master/makesd/for_1.0.7.alpha/make_sd_card.py
```
- Download the **make_ubuntu_sd.sh** script:

```
wget https://raw.githubusercontent.com/Ascend/tools/master/makesd/for_1.0.7.alpha/make_ubuntu_sd.sh
```

 NOTE

You can modify the following parameters in **make_sd_card.py** to configure the IP addresses of the USB NIC and NIC of the Atlas 200 DK:

- **NETWORK_CARD_DEFAULT_IP**: IP address of the NIC. Defaults to **192.168.0.2**.
- **USB_CARD_DEFAULT_IP**: IP address of the USB NIC. Defaults to **192.168.1.2**.

Step 6 Run the SD card preparation scripts.

1. Query the device name of the SD card USB as the **root** user:

```
fdisk -l
```

For example, the device name of the SD card USB is **/dev/sda**. The device name can be determined by removing and inserting the USB device.

2. Run the **make_sd_card.py** script.

```
python3 make_sd_card.py local /dev/sda
```

- **local**: The SD card is prepared in offline mode.
- **/dev/sda**: device name of the SD card USB.

The message shown in **Figure 4-1** indicates successful SD card preparation.

Figure 4-1 Message indicating successful SD card preparation



```
root@ascend:~# cd /home/ascend/mksd; python3 make_sd_card.py local /dev/sda
Begin to make SD Card...
Please make sure you have installed dependency packages:
  apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
Please input Y to continue, other to install them:
Step: Start to make SD Card. It need some time, please wait...
Command:
bash /home/ascend/mksd/make_ubuntu_sd.sh /dev/sda /home/ascend/mksd/ubuntu-18.04.4-server-arm64.iso 192.168.0.2 192.168.1.2 > /home/ascend/mksd/sd_card_making_log/make_ubuntu_sd.log
Make SD Card successfully!
```

 NOTE

If card preparation fails, check the log files in the **sd_card_making_log** folder in the current directory.

Step 7 After the card is successfully prepared, remove the SD card from the card reader and insert it into the card slot of the Atlas 200 DK.

Step 8 Power on the Atlas 200 DK.

NOTICE

- During the first power-on and boot process, Firmware upgrade is implemented. After the upgrade is complete, the system reboots automatically. You can install other components after the reboot.
- Do not power off the Atlas 200 DK during the first boot. Otherwise, the Atlas 200 DK may be damaged. After it is powered off, wait at least 2s before powering it on again.

For details about how to power on the Atlas 200 DK and the description of the LED indicator status after power-on, see [5.2 Powering on Atlas 200 DK](#).

----End

Exception Handling

After powering-on, if the Atlas 200 DK cannot be started properly (the indicator status is abnormal), perform the following steps to view the related logs:

Step 1 Power off the Atlas 200 DK.

Step 2 Remove the SD card from the Atlas 200 DK, insert the SD card into the card reader, and connect it to the Ubuntu server over the USB port.

Step 3 Run the following command as the **root** user to view the partition information of the SD card USB:

fdisk -l

The displayed information is as follows.

```
Disk /dev/sda: 29.7 GiB, 31914983424 bytes, 62333952 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1		2048	10487807	10485760	5G	83	Linux
/dev/sda2		10487808	12584959	2097152	1G	83	Linux
/dev/sda3		12584960	62333951	49748992	23.7G	83	Linux

Step 4 Mount the first partition of the SD card to the Ubuntu server.

1. Create an empty directory as the **root** user.

For example:

mkdir -p /home/sdinfo

2. Mount **/dev/sda1** to the **/home/sdinfo** directory.

mount /dev/sda1 /home/sdinfo

Step 5 Go to **/home/sdinfo**, that is, the Atlas 200 DK file system, to view the related logs in the **var/log/ascend_seclog** path.

cd /home/sdinfo

cd var/log/ascend_seclog/

The log file description is as follows:

- **operation.log**: operation log, recording the results of events, such as installation and upgrade.
The format is as follows: event type+event level+user ID+date+initiator address+access file name+command+result
- **ascend_install.log**: detailed O&M script log of installation and upgrade, from which operations and statuses can be viewed.
The format is as follows: component+date+log level+content
- **ascend_run_servers.log**: log recording the boot information of the Atlas 200 DK.

NOTICE

- If you cannot solve the problem, ask for help on the [Ascend Developer Zone](#) with the log file attached. Huawei engineers will provide technical support for you.
 - If the Atlas 200 DK fails to be started for three or more times, after this problem is solved, delete the **boot_fail_count** file in the **var/log/ascend_seclog/** directory. Otherwise, the Atlas 200 DK cannot be started properly.
-

----End

4.1.1.3 Creating an SD Card Without a Card Reader

This section describes how to prepare an SD card by short-circuiting the pins on Atlas 200 DK with a jumper cap or jumper wire without a card reader.

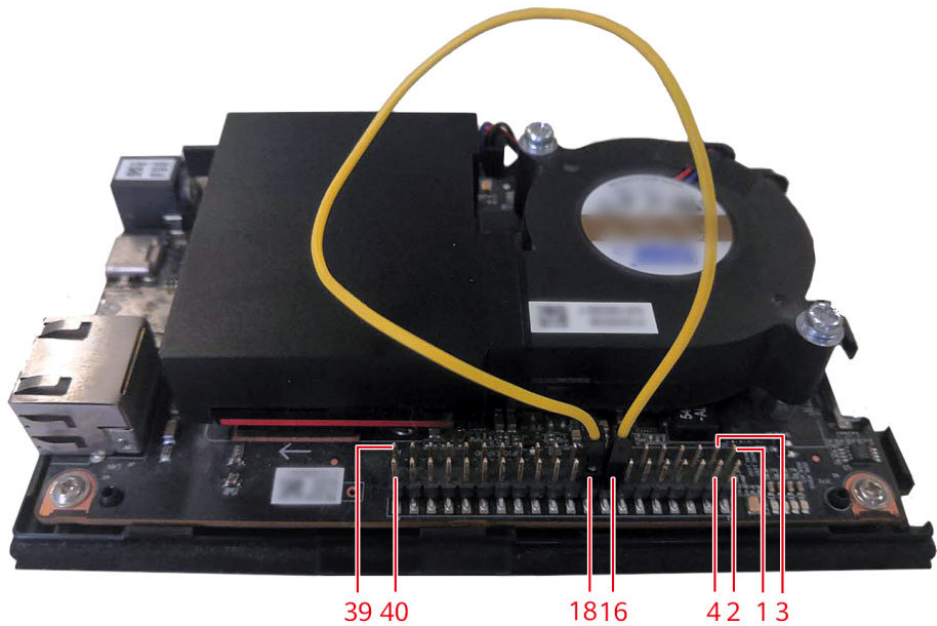
Hardware Preparation

1. Remove the top cover by referring to [3.1 Removing the Upper Case](#).
2. Place the jumper cap or jumper wire over pin 16 and pin 18 on the Atlas 200 DK, as shown in [Figure 4-2](#).

NOTICE

- Before performing this operation, power off the Atlas 200 DK. For details about power-off requirements, see [5.3 Powering off the Atlas 200 DK Developer Board](#).
 - Check the pins carefully. If incorrect pins are used, the Atlas 200 DK will be severely damaged.
 - The positions of pins 1, 2, and 40 are marked in white on the panel.
-

Figure 4-2 Installing a jumper wire



3. Connect the Atlas 200 DK to the Ubuntu server over the USB port.
4. Power on the Atlas 200 DK. For details, see [5.2 Powering on Atlas 200 DK](#).

Software Preparation

[Table 4-2](#) describes how to obtain the Driver packages and runfile of Atlas 200 DK and the Ubuntu OS image.

Table 4-2 Required files

Descri ption	File Name	Details	URL
Ubunt u OS image	ubuntu-18.04.xx- server-arm64.iso	OS image of the Atlas 200 DK. Ubuntu OS version: 18.04.4 or 18.04.5. Must be a server release for ARM hardware.	Link

Descri ption	File Name	Details	URL
Driver packag es and runfile of the Atlas 200 DK	Ascend310-driver- <i>{software version}</i> - ubuntu18.04.aarch6 4-minirc.tar.gz	Driver package, including OS peripheral software, AI software stack, maintenance and testing software, as well as drivers. During SD card preparation, Firmware information is obtained from the Driver package, so the Firmware component is not needed.	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.
	Ascend310- aicpu_kernels- <i>{software version}</i> - minirc.tar.gz	AI CPU OPP.	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.
	Ascend-acclib- <i>{software version}</i> - ubuntu18.04.aarch6 4-minirc.run	Ascend Computing Language Library (ACCLib) runfile, which is used to build and run AscendCL applications, enabling model loading and execution. During SD card creation, install the runfile as the HwHiAiUser user. The LD_LIBRARY_PATH environment variable will be automatically set in the .bashrc file of the HwHiAiUser user. export LD_LIBRARY_PATH=/ home/HwHiAiUser/ Ascend/acclib/lib64	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.

NOTICE

Keep the names of the downloaded files unchanged.

Procedure

- Step 1** Run the following commands on the Ubuntu server to install the qemu-user-static, binfmt-support, YAML, and cross compiler:

su - root

Update the sources:

apt-get update

Install the Python dependencies:

pip3 install pyyaml

apt-get install qemu-user-static binfmt-support python3-yaml gcc-aarch64-linux-gnu g++-aarch64-linux-gnu

- Step 2** Run the following command as the **root** user on the Ubuntu server to create a card creation project directory:

mkdir /home/ascend/mksd

The card creation project directory is user-defined.

- Step 3** Upload the obtained Ubuntu OS image package and Driver packages of the Atlas 200 DK to the card creation project directory (for example, **/home/ascend/mksd**).

- Step 4** Run the following commands in the card creation project directory (for example, **/home/ascend/mksd**) to download the card creation scripts:

- Download the **make_sd_card.py** script:
wget https://raw.githubusercontent.com/Ascend/tools/master/makesd/for_1.0.7.alpha/make_sd_card.py
- Download the **make_ubuntu_sd.sh** script:
wget https://raw.githubusercontent.com/Ascend/tools/master/makesd/for_1.0.7.alpha/make_ubuntu_sd.sh

 NOTE

You can modify the following parameters in **make_sd_card.py** to configure the IP addresses of the USB NIC and NIC of the Atlas 200 DK:

- **NETWORK_CARD_DEFAULT_IP**: IP address of the NIC. Defaults to **192.168.0.2**.
- **USB_CARD_DEFAULT_IP**: IP address of the USB NIC. Defaults to **192.168.1.2**.

- Step 5** Run the SD card preparation scripts.

1. Query the device name of the SD card USB as the **root** user:
fdisk -l

For example, the device name of the SD card USB is **/dev/sda**. The device name can be determined by removing and inserting the USB device.

2. Run the **make_sd_card.py** script.

python3 make_sd_card.py local /dev/sda

- **local**: The SD card is prepared in offline mode.
- **/dev/sda**: device name of the SD card USB.

The message shown in **Figure 4-3** indicates successful SD card preparation.

Figure 4-3 Message indicating successful SD card preparation

```
root@ascend@Probesk-600-64-PCI-MT:/home/ascend/mksd# python3 make_sd_card.py local /dev/sda
Begin to make SD Card...
Please make sure you have installed dependency packages:
  apt-get install -y qemu-user-static binfmt-support gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
Please input Y to continue, other to install them?
Step: Start to make SD Card. It need some time, please wait...
Command:
bash /home/ascend/mksd/make_ubuntu_sd.sh /dev/sda /home/ascend/mksd/ubuntu-18.04.4-server-arm64.iso 192.168.0.2 192.168.1.2 > /home/ascend/mksd/sd_card_making_log/make_ubuntu_sd.log
Make SD Card successfully!
```

NOTE

If card preparation fails, check the log files in the **sd_card_making_log** folder in the current directory.

Step 6 Power off the Atlas 200 DK. For details, see [5.3 Powering off the Atlas 200 DK Developer Board](#).

Step 7 Remove the jumper cap or jumper wire.

Step 8 Power on the Atlas 200 DK.

NOTICE

- During the first power-on and boot process, Firmware upgrade is implemented. After the upgrade is complete, the system reboots automatically. You can install other components after the reboot.
- Do not power off the Atlas 200 DK during the first boot. Otherwise, the Atlas 200 DK may be damaged. After it is powered off, wait at least 2s before powering it on again.

For details about how to power on the Atlas 200 DK and the description of the LED indicator status after power-on, see [5.2 Powering on Atlas 200 DK](#).

----End

Exception Handling

After powering-on, if the Atlas 200 DK cannot be started properly (the indicator status is abnormal), perform the following steps to view the related logs:

Step 1 Power off the Atlas 200 DK.

Step 2 Place the jumper cap or jumper wire over pin 16 and pin 18 on the Atlas 200 DK to use it as a USB device, as shown in [Hardware Preparation](#).

Step 3 Connect the Atlas 200 DK to the Ubuntu server over the USB port, and power on the Atlas 200 DK.

- Step 4** Run the following command as the **root** user to view the partition information of the SD card USB:

fdisk -l

The displayed information is as follows.

```
Disk /dev/sda: 29.7 GiB, 31914983424 bytes, 62333952 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000
```

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1		2048	10487807	10485760	5G	83	Linux
/dev/sda2		10487808	12584959	2097152	1G	83	Linux
/dev/sda3		12584960	62333951	49748992	23.7G	83	Linux

- Step 5** Mount the first partition of the SD card to the Ubuntu server.

1. Create an empty directory as the **root** user.

For example:

mkdir -p /home/sdinfo

2. Mount **/dev/sda1** to the **/home/sdinfo** directory.

mount /dev/sda1 /home/sdinfo

- Step 6** Go to **/home/sdinfo**, that is, the Atlas 200 DK file system, to view the related logs in the **var/log/ascend_seclog** path.

cd /home/sdinfo

cd var/log/ascend_seclog/

The log file description is as follows:

- **operation.log**: operation log, recording the results of events, such as installation and upgrade.
The format is as follows: event type+event level+user ID+date+initiator address+access file name+command+result
- **ascend_install.log**: detailed O&M script log of installation and upgrade, from which operations and statuses can be viewed.
The format is as follows: component+date+log level+content
- **ascend_run_servers.log**: log recording the boot information of the Atlas 200 DK.

NOTICE

- If you cannot solve the problem, ask for help on the [Ascend Developer Zone](#) with the log file attached. Huawei engineers will provide technical support for you.
- If the Atlas 200 DK fails to be started for three or more times, after this problem is solved, delete the **boot_fail_count** file in the **var/log/ascend_seclog/** directory. Otherwise, the Atlas 200 DK cannot be started properly.

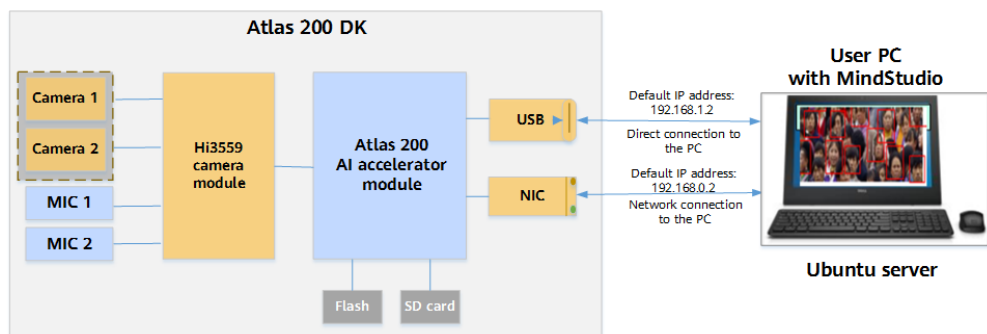
----End

4.1.2 Connecting the Atlas 200 DK to the Ubuntu Server

Scenario Description

You can use a USB port or network cable to connect the Atlas 200 DK to the Ubuntu server, as shown in [Figure 4-4](#).

Figure 4-4 Connection between the Atlas 200 DK and Mind Studio



The Atlas 200 DK can be connected to the Ubuntu server in the following modes:

- Connects to the Ubuntu server over the USB port. For details, see [Connecting to the Ubuntu Server over the USB Port](#).
In this mode, the Atlas 200 DK cannot access the Internet. Only communication with the Ubuntu server is available.
- Connects the Atlas 200 DK to the router using a network cable and then to the Ubuntu server through the network. For details, see [\(Recommended\) Connecting to the Ubuntu Server Through the Network by Connecting to a Router Using a Network Cable](#).
In this mode (recommended), the Atlas 200 DK can directly access the Internet.
- Connects the Atlas 200 DK to the network port of the Ubuntu server using a network cable. For details, see [Connecting to the Ubuntu Server by Using a Network Cable](#).
In this mode, the Atlas 200 DK cannot access the Internet. Only communication with the Ubuntu server is available.

 NOTE

If you have changed the IP address of the Atlas 200 DK and that of the USB virtual NIC on the Ubuntu server to the same network segment during SD card preparation, skip the following steps of changing the IP address of the USB virtual NIC on the Ubuntu server.

Connecting to the Ubuntu Server over the USB Port

In this mode, the default IP address of the USB virtual NIC of the Atlas 200 DK is **192.168.1.2**. Therefore, the IP address of the USB virtual NIC on the Ubuntu server needs to be changed to **192.168.1.x** (the value of *x* can be 0, 1, or 3–254).

The following describes how to configure the IP address of the USB virtual NIC on the Ubuntu server manually and by using a script.

NOTICE

If the Ubuntu server is installed in a VM running Windows on the host, you need to install the USB virtual NIC driver on the Windows host by referring to [5.10 Installing the Windows USB Network Adapter Driver](#). Otherwise, the USB virtual NIC of the Atlas 200 DK cannot be identified by the Ubuntu server.

If the Atlas 200 DK has been connected to the Ubuntu server using a USB port, perform the following steps for IP address configuration.

- Configuring the IP address by using a script:
 - a. Download **configure_usb_ethernet.sh** from https://github.com/Huawei-Ascend/tools/tree/master/configure_usb_ethernet/for_1.7x.0.0 to any directory on the Ubuntu server, for example, **/home/ascend/config_usb_ip/**.

NOTICE

You can use the script only when configuring the IP address of the USB NIC for the first time. After the IP address of the USB NIC is configured, you can manually change the IP address by referring to [Configuring the IP address manually](#).

- b. Go to the directory where the script for configuring the IP address of the USB virtual NIC is located as the **root** user, for example, **/home/ascend/config_usb_ip**.
- c. Configure the IP address of the USB virtual NIC:
bash configure_usb_ethernet.sh -s ip_address
Specify the static IP address of the USB NIC. If **bash configure_usb_ethernet.sh** is run directly, the default IP address **192.168.1.166** is used.
 - If there are multiple USB NICs, run the **ifconfig** command to query the names of the USB NICs, and remove and insert the Atlas 200 DK to determine the USB NIC name of the Atlas 200 DK. The Atlas 200 DK is identified as a USB virtual NIC by the Ubuntu server. Run the following command to configure the IP address:

```
bash configure_usb_ethernet.sh -s usb_nic_name ip_address
```

usb_nic_name: name of the USB virtual NIC

ip_address: IP address to be configured

For example, to set the IP address of the USB virtual NIC on the Ubuntu server to **192.168.1.223**, run the following command:

```
bash configure_usb_ethernet.sh -s enp0s20f0u8 192.168.1.223
```

After the configuration is complete, run the **ifconfig** command to check whether the IP address takes effect.

- Configuring the IP address manually:

- a. Log in to the Ubuntu server as a common user and run the following command to switch to the **root** user:

```
su - root
```

- b. Obtain the name of the USB virtual NIC.

```
ifconfig -a
```

If there are multiple USB NICs, remove and insert the Atlas 200 DK to determine the required one.

- c. Add the static IP address of the USB NIC to the **/etc/netplan/01-netcfg.yaml** file.

Run the following command to open the **01-netcfg.yaml** file:

```
vi /etc/netplan/01-netcfg.yaml
```

Add the network configuration of the USB NIC at the **ethernets** layer. For example, if the USB NIC name is **enp0s20f0u4** and the static IP address is **192.168.1.223**, the configuration should be as follows:

```
ethernets:
  ...
  enp0s20f0u4:
    dhcp4: no
    addresses: [192.168.1.223/24]
    gateway4: 192.168.0.1
    nameservers:
      addresses: [255.255.0.0]
```

Enter **:wq!** to save the change and exit.

- d. Restart the network service.

netplan apply

After the reboot, run the **ifconfig** command to check whether the IP address of the USB NIC **enp0s20f0u4** takes effect.

(Recommended) Connecting to the Ubuntu Server Through the Network by Connecting to a Router Using a Network Cable

In this mode, the DHCP function needs to be enabled on the router, which will automatically assign an IP address to the Atlas 200 DK. The IP address obtaining mode of the NIC on the Atlas 200 DK should be changed to DHCP accordingly. You need to connect the Atlas 200 DK to the Ubuntu server using a USB cable, and then log in to the Atlas 200 DK in SSH mode through the Ubuntu server to change the mode of obtaining the virtual NIC IP address.

Assume that you have connected the Atlas 200 DK to the router using a network cable and enabled the DHCP function of the router. To connect to the Ubuntu server, perform the following steps:

1. Connect the Atlas 200 DK to the Ubuntu server using a USB port, and configure the IP address of the USB NIC of the Ubuntu server. For details, see [Connecting to the Ubuntu Server over the USB Port](#).
2. Change the IP address obtaining mode of the NIC on the Atlas 200 DK to DHCP.
 - a. Log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode on the Ubuntu server.
ssh HwHiAiUser@192.168.1.2
The default IP address of the USB NIC on the Atlas 200 DK is **192.168.1.2**.
The default login password of the **HwHiAiUser** user is **Mind@123**.
 - b. Switch to the **root** user and open the network configuration file.
su root
vi /etc/netplan/01-netcfg.yaml
 - c. Change the IP address obtaining mode of eth0 to DHCP.
Modify the configuration of eth0 as follows.

```
eth0:
  dhcp4: true
  addresses: []
  optional: true
```
 - d. Save the modifications and exit.
:wq
3. Restart the network service.
netplan apply
The Atlas 200 DK can access the Internet now.
4. Run the **ifconfig** command to obtain the IP address of eth0. This IP address and that of the USB NIC can be used to communicate with the Ubuntu server.

Connecting to the Ubuntu Server by Using a Network Cable

In this mode, the IP address of the Ubuntu server needs to be changed to **192.168.0.x** (the value of *x* can be 0, 1, or 3–254).

NOTE

- The default IP address of the NIC on the Atlas 200 DK is **192.168.0.2**, and the subnet mask is 24 bits.
- After the network port on the Atlas 200 DK is connected to a network cable, if the yellow ACT indicator blinks, data is being transmitted. When the network port of the Atlas 200 DK accesses the GE network, the green LINK indicator is on. When the network port of the Atlas 200 DK accesses the 100M/10M Ethernet network, the LINK indicator is off, which is normal.

Perform the following steps:

1. Log in to the Ubuntu server as a common user and run the following command to switch to the **root** user:
su - root
2. Configure an IP address for the virtual NIC to communicate with the Atlas 200 DK.
For example, to configure the virtual static IP address of **eth0:1**, run the following command:

```
ifconfig eth0:1 192.168.0.223 netmask 255.255.0.0 up
```

Follow-up Operations

After the Atlas 200 DK is connected to the Ubuntu server, you can determine whether to reboot the OS on the Atlas 200 DK or power off the Atlas 200 DK based on the Atlas 200 DK LED indicators. For details, see [Table 5-2](#).

NOTICE

Restart or power off the server or Atlas 200 DK with caution, especially when the Atlas 200 DK is being upgraded.

4.2 Installing the Development Environment

4.2.1 Obtaining Software Packages

Before installing the software, obtain the following software packages. *{version}* indicates the software package version, which must be the same as the actual version.

Table 4-3 Software packages

Name	Software Package	How to Obtain	Function
Development kit	Ascend-Toolkit- <i>{version}</i> -x86_64-linux_gcc7.3.0.run	Link 1. Choose AI Developer Kit from Product Series. 2. Choose Atlas 200 DK AI Developer Kit from Product Model. 3. Choose 1.0.7.alpha from Version.	• It is used for application development, operator customization, and model conversion. The development kit contains the library files required for developing applications and auxiliary development tools such as the ATC model conversion tool. • Unified installation for two architectures: The development environment uses

Name	Software Package	How to Obtain	Function
	Ascend-Toolkit- {version}-arm64- linux_gcc7.3.0.run		the x86 architecture, but the operating environment uses the ARM architecture. Therefore, an ARM64 development kit needs to be installed for cross compilation of applications.

4.2.2 Configuring Ubuntu x86

Checking the umask of the root User

1. Log in to the installation environment as the **root** user.
2. Check the **umask** value of the **root** user.

```
umask
```

3. If the **umask** value is not **0022**, append "umask 0022" to the file and save the file:

```
vi ~/.bashrc  
source ~/.bashrc
```

Creating an Installation User

If a non-root user is used to install the development kit, you need to create a non-root user. Perform the creation as the **root** user.

1. Create a non-root user.

```
useradd -d /home/username -m username
```


username is user-defined.
2. Set the password of the non-root user.

```
passwd username
```

NOTE

The password validity period is 90 days. You can change the validity period in the `/etc/login.defs` file or using the **chage** command. For details, see [5.11 Setting User Account Validity Period](#).

Configuring Permissions for the Installation User

If a non-root user is used for installation, you need to configure the permission for the installation user.

Before installing the development kit, you need to download the dependencies, which require **sudo apt-get** permission. Run the following commands as the **root** user:

1. Open the **/etc/sudoers** file:

```
chmod u+w /etc/sudoers  
vi /etc/sudoers
```

2. Add the following content below **# User privilege specification** of the file:

```
username ALL=(ALL:ALL) NOPASSWD:SETENV:/usr/bin/apt-get, /usr/bin/pip, /bin/tar, /bin/  
mkdir, /bin/rm, /bin/sh, /bin/cp, /bin/bash, /usr/bin/make install, /bin/ln -s /usr/local/python3.7.5/bin/  
python3 /usr/bin/python3.7, /bin/ln -s /usr/local/python3.7.5/bin/pip3 /usr/bin/pip3.7, /bin/ln -s /usr/  
local/python3.7.5/bin/python3 /usr/bin/python3.7.5, /bin/ln -s /usr/local/python3.7.5/bin/pip3 /usr/bin/  
pip3.7.5, /usr/bin/unzip
```

Replace **username** with the name of the common user who executes the installation script.

NOTE

Ensure that the last line of the **/etc/sudoers** file is **#includedir /etc/sudoers.d**. Otherwise, add it manually.

3. Run the **:wq!** command to save the file.
4. Run the following command to revoke the write permission on the **/etc/sudoers** file:

```
chmod u-w /etc/sudoers
```

Checking the Source Validity

Development kit installation requires the download of related dependencies. Ensure that the installation environment can be connected to the network.

Run the following command as the **root** user to check whether the source is valid:

```
apt-get update
```

If an error is reported during command execution or dependency installation, check whether the network connection is normal, or replace the source in the **/etc/apt/sources.list** file with an available source or use an image source. For details about how to configure a network proxy, see [5.13 Configuring a System Network Proxy](#).

Installing Dependencies

NOTE

- If you perform the following steps as the **root** user to install Python and its dependencies, delete **sudo** from the commands.
- If you install Python and its dependencies as a non-root user, run the **su - username** command to switch to the non-root user and perform the following steps.

Step 1 Check whether the Python dependencies and GCC software are installed.

Run the following commands to check whether the dependencies such as GCC, Make, and Python are installed:

```
gcc --version  
g++ --version  
make --version  
cmake --version  
dpkg -l zlib1g| grep zlib1g| grep ii  
dpkg -l zlib1g-dev| grep zlib1g-dev| grep ii
```

```
dpkg -l libsqlite3-dev| grep libsqlite3-dev| grep ii
dpkg -l openssl| grep openssl| grep ii
dpkg -l libssl-dev| grep libssl-dev| grep ii
dpkg -l libffi-dev| grep libffi-dev| grep ii
dpkg -l unzip| grep unzip| grep ii
dpkg -l pciutils| grep pciutils| grep ii
dpkg -l net-tools| grep net-tools| grep ii
dpkg -l libblas-dev| grep libblas-dev| grep ii
dpkg -l gfortran| grep gfortran| grep ii
dpkg -l libblas3| grep libblas3| grep ii
dpkg -l libopenblas-dev| grep libopenblas-dev| grep ii
```

If the following information is displayed, the installation is complete. Go to the next step.

```
gcc (Ubuntu 7.3.0-3ubuntu1~18.04) 7.3.0
g++ (Ubuntu 7.3.0-3ubuntu1~18.04) 7.3.0
GNU Make 4.1
cmake version 3.10.2
zlib1g:arm64 1:1.2.11.dfsg-0ubuntu2 arm64 compression library - runtime
zlib1g-dev:arm64 1:1.2.11.dfsg-0ubuntu2 arm64 compression library - development
libsqlite3-dev:arm64 3.22.0-1ubuntu0.3 arm64 SQLite 3 development files
openssl 1.1.1-1ubuntu2.1~18.04.6 arm64 Secure Sockets Layer toolkit - cryptographic utility
libssl-dev:arm64 1.1.1-1ubuntu2.1~18.04.6 arm64 Secure Sockets Layer toolkit - development files
libffi-dev:arm64 3.2.1-8 arm64 Foreign Function Interface library (development files)
unzip 6.0-21ubuntu1 amd64 De-archiver for .zip files
pciutils 1:3.5.2-1ubuntu1 arm64 Linux PCI Utilities
net-tools 1.60+git20161116.90da8a0-1ubuntu1 arm64 NET-3 networking toolkit
libblas-dev:arm64 3.7.1-4ubuntu1 arm64 Basic Linear Algebra Subroutines 3, static library
gfortran 4:7.4.0-1ubuntu2.3 arm64 GNU Fortran 95 compiler
libblas3:arm64 3.7.1-4ubuntu1 arm64 Basic Linear Algebra Reference implementations, shared library
libopenblas-dev:arm64 0.2.20+ds-4 arm64 Optimized BLAS (linear algebra) library (development files)
```

Otherwise, run the following command to install the software. You can change the following command to install only some of them as required.

```
sudo apt-get install -y gcc g++ make cmake zlib1g zlib1g-dev libsqlite3-dev openssl libssl-dev libffi-dev
unzip pciutils net-tools libblas-dev gfortran libblas3 libopenblas-dev
```

Step 2 Check whether the Python development environment is installed.

The development kit depends on the Python environment. Run the **python3.7.5 --version** and **pip3.7.5 --version** commands to check whether they have been installed. If the following information is displayed, they have been installed. Go to the next step.

```
Python 3.7.5
pip 19.2.3 from /usr/local/python3.7.5/lib/python3.7/site-packages/pip (python 3.7)
```

Otherwise, use the following procedure to install Python 3.7.5:

1. Run the **wget** command to download the source code package of Python 3.7.5 to any directory of the installation environment. The command is as follows:

```
wget https://www.python.org/ftp/python/3.7.5/Python-3.7.5.tgz
```
2. Run the following command to go to the download directory and decompress the source code package:

```
tar -zxvf Python-3.7.5.tgz
```
3. Go to the decompressed folder and run the following configuration, build, and installation commands:

```
cd Python-3.7.5
./configure --prefix=/usr/local/python3.7.5 --enable-shared
make
sudo make install
```

The **--prefix** parameter specifies the Python installation path. You can change it based on the site requirements. The **--enable-shared** parameter is used to compile the **libpython3.7m.so.1.0** dynamic library.

This document uses **--prefix=/usr/local/python3.7.5** as an example. After the configuration, compilation, and installation commands are executed, the installation package is output to the **/usr/local/python3.7.5** directory, and the **libpython3.7m.so.1.0** dynamic library is output to the **/usr/local/python3.7.5/lib/libpython3.7m.so.1.0** directory.

4. Check whether **libpython3.7m.so.1.0** exists in **/usr/lib64** or **/usr/lib**. If yes, skip this step or back up the **libpython3.7m.so.1.0** file provided by the system and run the following command:

Copy the compiled file **libpython3.7m.so.1.0** to **/usr/lib64**:

```
sudo cp /usr/local/python3.7.5/lib/libpython3.7m.so.1.0 /usr/lib64
```

When the following information is displayed, enter **y** to overwrite the **libpython3.7m.so.1.0** file provided by the system.

```
cp: overwrite 'libpython3.7m.so.1.0'?y
```

If the **/usr/lib64** directory does not exist in the environment, copy the file to the **/usr/lib** directory.

```
sudo cp /usr/local/python3.7.5/lib/libpython3.7m.so.1.0 /usr/lib
```

Replace the path of the **libpython3.7m.so.1.0** file as required.

5. Run the following commands to set the soft link:

```
sudo ln -s /usr/local/python3.7.5/bin/python3 /usr/bin/python3.7
sudo ln -s /usr/local/python3.7.5/bin/pip3 /usr/bin/pip3.7
sudo ln -s /usr/local/python3.7.5/bin/python3 /usr/bin/python3.7.5
sudo ln -s /usr/local/python3.7.5/bin/pip3 /usr/bin/pip3.7.5
```

If a message indicating that the link already exists is displayed during the command execution, run the following command to delete the existing link and run the command again:

```
sudo rm -rf /usr/bin/python3.7.5
sudo rm -rf /usr/bin/pip3.7.5
sudo rm -rf /usr/bin/python3.7
sudo rm -rf /usr/bin/pip3.7
```

6. After the installation is complete, run the following commands to check the installation version. If the required version information is displayed, the installation is successful.

```
python3.7.5 --version
pip3.7.5 --version
```

Step 3 Install the Python 3 development environment.

Before the installation, run the **pip3.7.5 list** command to check whether the dependencies have been installed. If yes, skip this step. If not, run the following command to install the dependencies: (If only some of the software is not installed, modify the following command to install selected software only.) The Model Accuracy Analyzer in the development kit depends on protobuf and SciPy. Profiling depends on protobuf, grpcio, grpcio-tools, and requests.

If you install Python and its dependencies as a non-root user, add **--user** at the end of each command in this step to ensure that the installation is successful.

Example command: **pip3.7.5 install attrs --user**

```
pip3.7.5 install attrs
pip3.7.5 install psutil
pip3.7.5 install decorator
pip3.7.5 install numpy
```

```
pip3.7.5 install protobuf==3.11.3
pip3.7.5 install scipy
pip3.7.5 install sympy
pip3.7.5 install cffi
pip3.7.5 install grpcio
pip3.7.5 install grpcio-tools
pip3.7.5 install requests
```

During the command execution, if the network connection fails and the message "Could not find a version that satisfies the requirement xxx" is displayed, rectify the fault by referring to [6.3 What Do I Do If "Could not find a version that satisfies the requirement xxx" Is Displayed When pip3.7.5 install Is Run?](#).

----End

4.2.3 Installing the Development Kit

Prerequisites

- Prepare for the installation by referring to [4.2.2 Configuring Ubuntu x86](#).
- You have obtained the development kit software packages **Ascend-Toolkit-*{version}*-x86_64-linux_gcc7.3.0.run** and **Ascend-Toolkit-*{version}*-arm64-linux_gcc7.3.0.run**.

Procedure

Step 1 Log in to the installation environment as the installation user (same as the user in [Installing Dependencies](#)) of the software package.

Step 2 Upload the development kit package applicable to the ARM and x86 architectures to any path (for example, **/home**) in the installation environment, and go to the path where the package is stored.

Step 3 Run the following commands to add the execute permission and verify the consistency and integrity:

```
chmod +x *.run
```

```
./*.run --check
```

* indicates the name of the development kit package. Replace it with the actual name.

Step 4 Install the software.

- The installation path is specified.

```
./*.run --install --install-path={path}
```

{path} indicates the specified installation path. Replace it with the actual path.

- The installation path is not specified.

```
./*.run --install
```

For details about the installation path, see [Installation Path Description](#).

 NOTE

- The development kits of multiple versions can be installed by different users in the same development environment. The users must be in the same group as the Driver running user. If the owner groups are different, add the users to the group of the Driver running user.
- If the installation is performed by the **root** user, **do not to specify the installation path in the directory of a non-root user**. Otherwise, the **root** user file may be replaced by a non-root user for privilege escalation.
- The AI CPU operator package in the development kit must be installed as the **root** user. If you perform the installation as a non-root user, the AI CPU operator package (**Ascend310-aicpu_kernels-*{version}*.run**) will be automatically decompressed to the ***\${install_path}/ascend-toolkit/{version}/xxx-linux*** directory during the installation. You need to add **sudo** before the following command or switch to the **root** user to install the operator package separately. Go to the ***\${install_path}/ascend-toolkit/{version}/xxx-linux*** directory and run the following command to perform the installation:

```
./Ascend310-aicpu_kernels-{version}.run --full
```

The AI CPU does not support the specified installation path and shares the installation path of the driver.
- The **--quiet** option is not supported when the development kit **Ascend-Toolkit-*{version}*-*{arch}*-linux_*{gccversion}*.run** is installed in an x86 system.
- If the following information is displayed during the installation of the development kit **Ascend-Toolkit-*{version}*-*{arch}*-linux_*{gccversion}*.run** in the x86 system, asking you whether to perform a hot reset, enter **n**. After the installation is complete, restart the OS for the setting to take effect. In the current version, only **n** is supported.
The installation of aicpu_kernels needs to restart the device to take effect, do you want to hot_reset the device? [y/n] n
- For more installation modes, see [5.14 Parameters](#).

----End

Installation Path Description

- If you need to specify the installation path, you need to create it first. For example, if the installation path is **/home/work**, run the **mkdir -p /home/work** command to create an installation path and then specify the path to install the software.
- If you do not specify an installation path, the software is installed in the default path. For details about the default paths, see [Table 4-4](#).

Table 4-4 Software package installation paths

Item	Path
Default installation path of a software package	• root user: <i>/usr/local/Ascend/\${package_name}/{version}/{arch-linux-gccversion}</i> • Non-root user: <i>\${HOME}/Ascend/\${package_name}/{version}/{arch-linux-gccversion}</i>
Installation log path	<i>\${install_path}/\${package_name}/{version}/{arch-linux-gccversion}/ascend_install.log</i>

Item	Path
Path for recording information such as the software package version, CPU architecture, GCC version, and installation path after the installation	\${install_path}/\${package_name}/{version}/{arch-linux-gccversion}/ascend-\${package_name}_install.info

Table 4-5 describes the variables in the software package installation paths listed in **Table 4-4**.

Table 4-5 Variable description

Parameter	Description
\${package_name}	Software package directory, which is named after a software package.
{version}	Version directory, which is named after the version number of the software package. NOTE Multiple versions can be installed, which are differentiated by version numbers.
{arch-linux_gccversion}	Architecture directory, which is named after the combination of the CPU architecture, Linux branch, and GCC version of the software package. NOTE Multiple versions can be installed.
\${HOME}	Home directory of the current user.
\${install_path}	Software package installation path.

4.2.4 Deploying the Media Module

If an external camera is used to collect source data for AI application development, related header file and library file need to be deployed in the development environment.

The procedure is as follows:

- Step 1** Upload **Ascend310-driver-*{software version}*-ubuntu18.04.aarch64-minirc.tar.gz** to any directory in the development environment as the development environment installation user.

For example, if the development environment installation user is **HwHiAiUser**, upload the package to the **/home/HwHiAiUser/software** directory.

Step 2 Decompress the Driver package as the installation user.

```
tar zxvf Ascend310-driver-{software version}-ubuntu18.04.aarch64-  
minirc.tar.gz
```

The header file and library file related to the Media module are stored in the extracted **driver** directory.

- **driver/peripheral_api.h**: header file of the Media module. For details, see [Media API Reference](#).

NOTICE

- The APIs of the Media module are based on the C language.
- If the Raspberry Pi v2.1 camera is used, the camera frame rates (FPS) can range from 1 to 20.
- If the Raspberry Pi v1.3 camera is used, the camera frame rates (FPS) can range from 1 to 15.

-
- **driver/libmedia_mini.so**: library file of the Media module.

Include the preceding header file and link the library file when developing applications using the Media module.

----End

4.2.5 Configuring the Cross-Compilation Environment

If the architecture of the development environment is different from that of the operating environment, you need to install the cross-compiler in the development environment. For details, see [Table 4-6](#).

Table 4-6 Cross-compiler installation

Development Environment Architecture	Operating Environment Architecture	Compilation Environment Configuration
x86_64	aarch64	Run the aarch64-linux-gnu-g++ --version command in the development environment as the installation user of the software package to check whether the software package has been installed. If the software package has been installed, skip this step. An example of the installation command is as follows. Run the command based on the actual situation. sudo apt-get install g++-aarch64-linux-gnu

4.2.6 Installing MindStudio

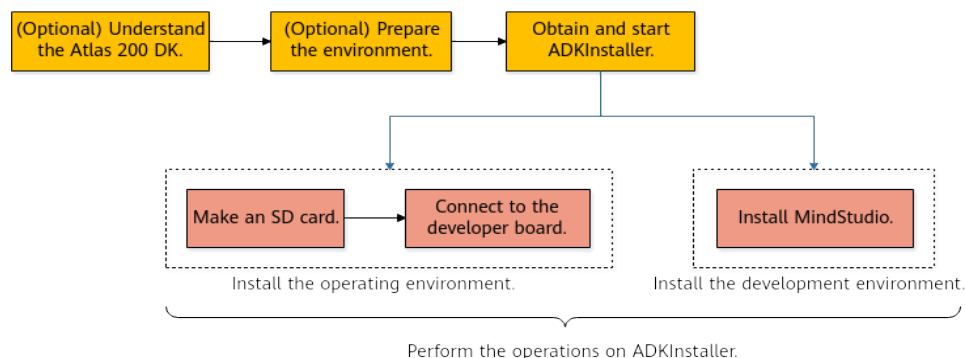
MindStudio is a Huawei AI full-stack integrated development environment (IDE) oriented to in-house Ascend AI processors. It provides functions such as network model migration, application development, inference execution, and custom operator development. Developers can perform end-to-end development on MindStudio, covering project management, building, debugging, running, and performance profiling, greatly enhancing the development efficiency. For details, see the [MindStudio User Guide](#).

4.3 Installing the Development Environment and Operating Environment

On the Ubuntu x86 server, use ADKInstaller to install the development environment and create a bootable SD card for the Atlas 200 DK. If the msInstaller tool cannot be used to install the environment, manually install the environment.

Environment Setup Process

The following figure shows the functions of the ADKInstaller tool and the overall process of setting up the environment.

Figure 4-5 Environment setup process

Installation Preparations

The Ubuntu x86 OS dependencies have been installed.

Obtaining and Starting ADKInstaller

Step 1 Obtain ADKInstaller.

1. Log in to the Ubuntu OS as a common user.
2. Download the ADKInstaller tool package to any directory in the system, for example, **/home/HwHiAiUser**.

```
wget https://obs-book.obs.cn-east-2.myhuaweicloud.com/temp/ADKInstaller.0.1.1.tar.gz
```

3. Decompress the software package and go to the directory where the decompressed files are stored.

```
tar -xzf ADKInstaller.0.1.1.tar.gz
```

```
cd ADKInstaller
```

4. Switch to the **root** user and run the following commands to enable the sudo permission for the current user:

```
su root
```

```
./add_sudo.sh username
```

```
exit
```

NOTE

- Replace *username* with the user name of the common user for installing the Atlas 200 DK development environment, for example, **ascend**.
- You need to temporarily enable the sudo permission for the current user. After the environment is installed, you can run the **./del_sudo.sh username** command as the **root** user to delete the sudo permission of the current user.

Step 2 Start ADKInstaller.

Run the following command as a common user in the Ubuntu system to start ADKInstaller:

```
./ADKInstaller
```

Figure 4-6 shows the page displayed after ADKInstaller is started.

- User: common user who starts ADKInstaller. If you log in as the **root** user, the software cannot be started.
- Password: password of the user who starts ADKInstaller.
- **Language:** English by default.

Figure 4-6 Login page

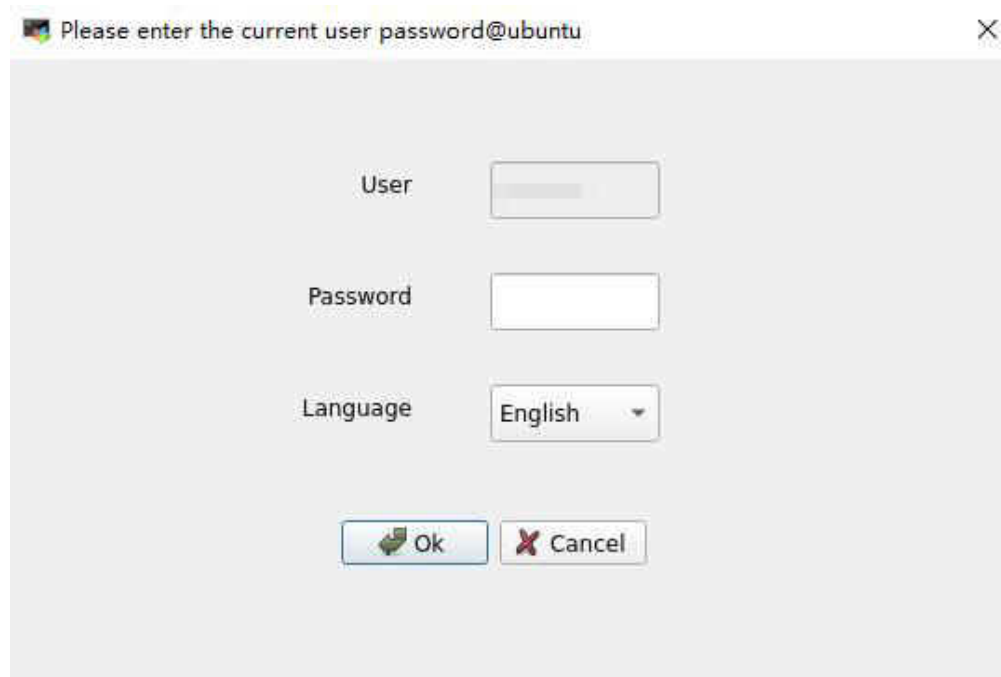
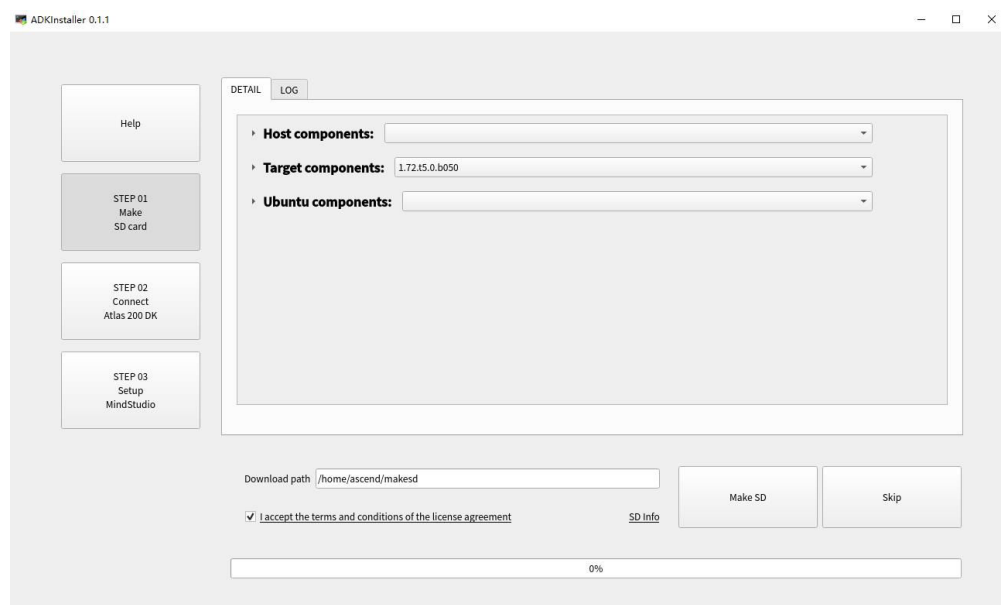


Figure 4-7 shows the page displayed after the login is successful.

Figure 4-7 ADKInstaller home page



----End

Installing the Operating Environment

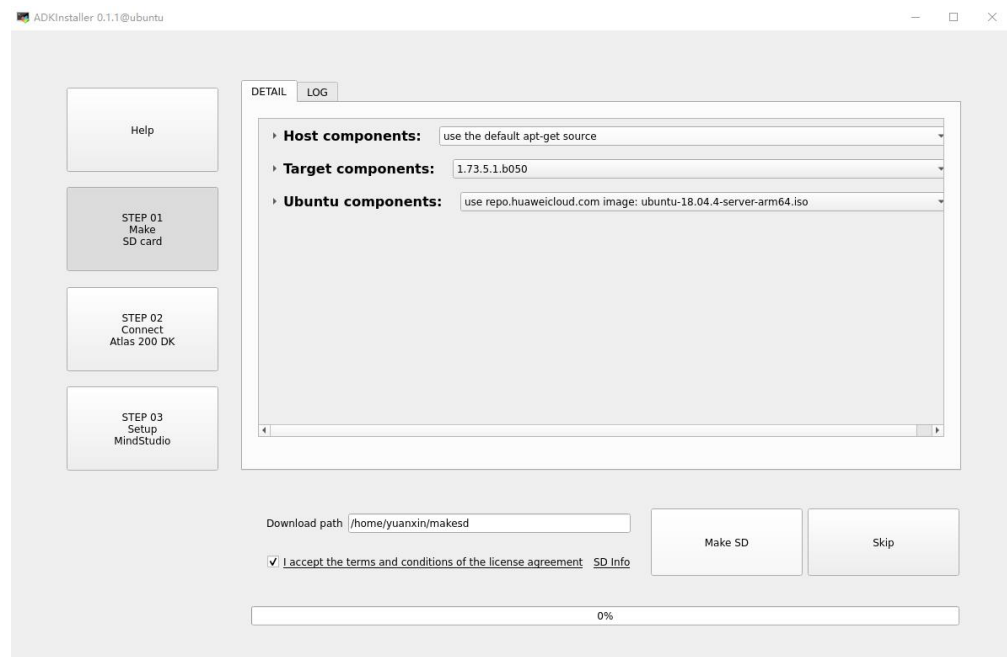
Step 1 Make an SD card.

NOTE

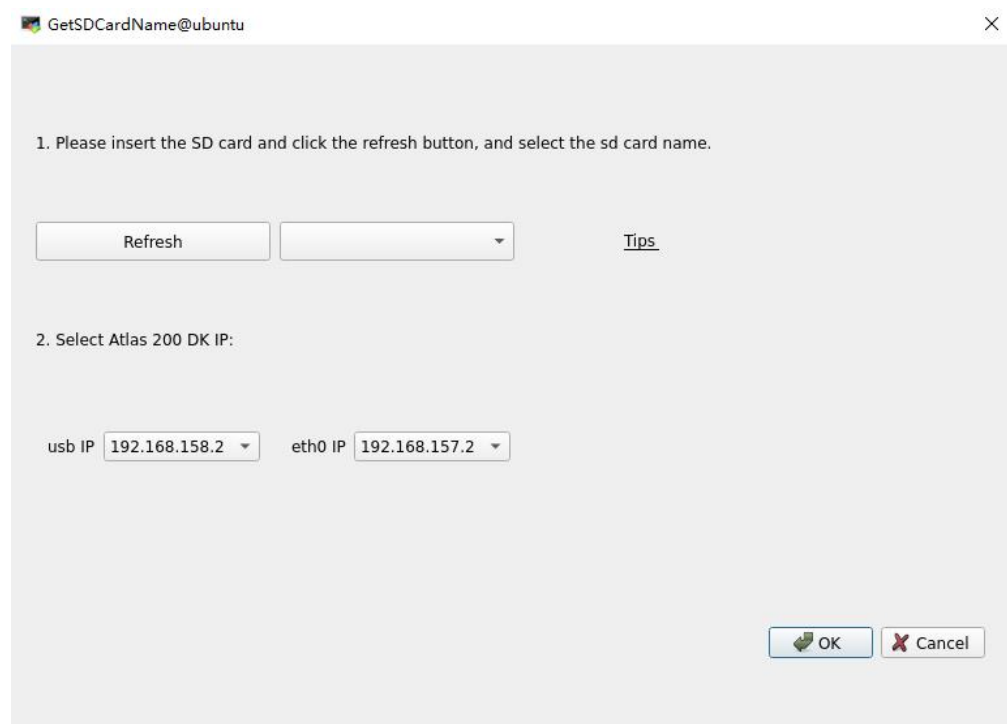
- The SD card is used to create the system boot disk for the Atlas 200 DK.
 - If you have a prepared SD card (the Atlas 200 DK can be started properly using this SD card), click **Skip** to skip this step and directly perform subsequent steps to upgrade the Atlas 200 DK.
1. On the **STEP 01 Make SD Card** page, select **I accept the terms and conditions of the license agreement** and click **OK** as prompted.
 2. On the page shown in [Figure 4-8](#), set the following parameters:
 - **Host components:** Select the installation source. You need to install the OS dependency (on the Ubuntu x86 server) when creating the SD card.
 - **Target components:** Select the installation source for installing the OS dependency on the SD card.
 - **Ubuntu components:** Ubuntu ISO installed on the SD card.
 - **Download Path:** Path for downloading files on the Ubuntu x86 server.
 - **SD INFO:** You can hover the mouse pointer over **SD INFO** to view the IP addresses of the USB NIC and physical NIC of the developer board for which the SD card is to be made. The default IP address of the USB NIC is 192.168.158.2, and the default IP address of the physical NIC is 192.168.157.2.

NOTE

- If the **apt-get update** command has not been executed on the Ubuntu server before ADKInstaller is used, you are advised to manually run the **apt-get update** command in the CLI. Otherwise, the apt dependency may fail to be installed. For details, see <https://gitee.com/lovingascend/ADKInstaller/blob/master/FAQ.md>.
- Users outside China are advised to select the default source, that is, use the default apt-get source.
- Users in China are advised to select Tsinghua source, that is, use the <https://mirrors.tuna.tsinghua.edu.cn/ubuntu/apt-get> source.

Figure 4-8 Making a card

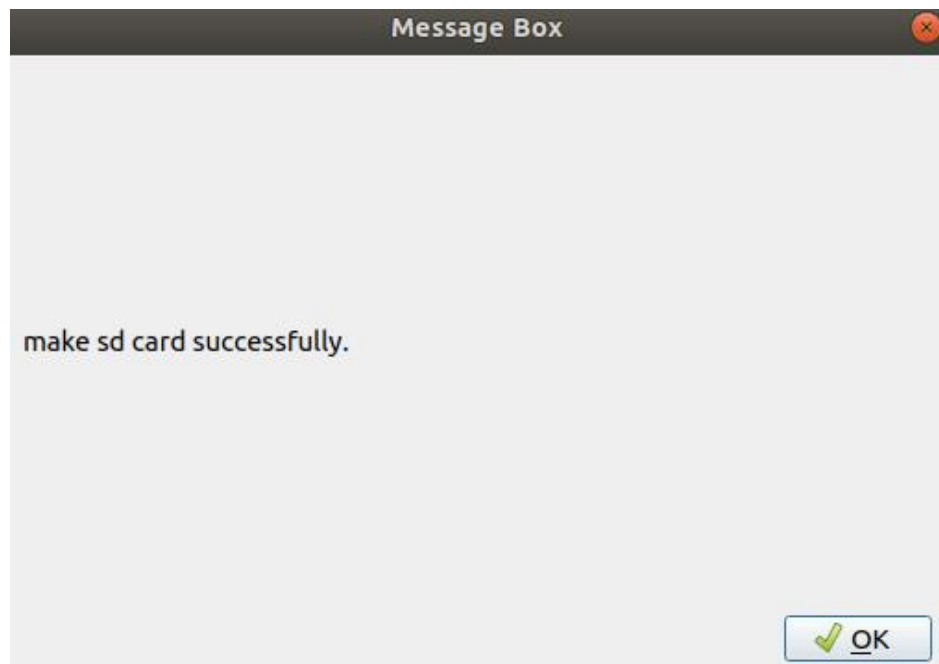
3. Click **Make SD** on the lower right of the page. After the component is downloaded, the page shown in [Figure 4-9](#) is displayed.

Figure 4-9 Settings of SD card making

- **usb IP:** IP address of the USB NIC
 - **eth0 IP:** IP address of the physical NIC
4. Insert a card reader with an SD card into the PC, click **Refresh**, and select the name of the SD card.

5. Select the USB NIC IP address (192.168.158.2 by default) and physical NIC IP address (eth0 IP address) of the Atlas 200 DK after card making.
The physical NIC IP address will be bound to the USB NIC IP address. The physical NIC IP address is automatically updated with the USB NIC IP address.
6. Click **OK** to start making the SD card.
7. After the card is made, a dialog box shown in [Figure 4-10](#) is displayed, indicating that the card is made successfully. Click **OK**.

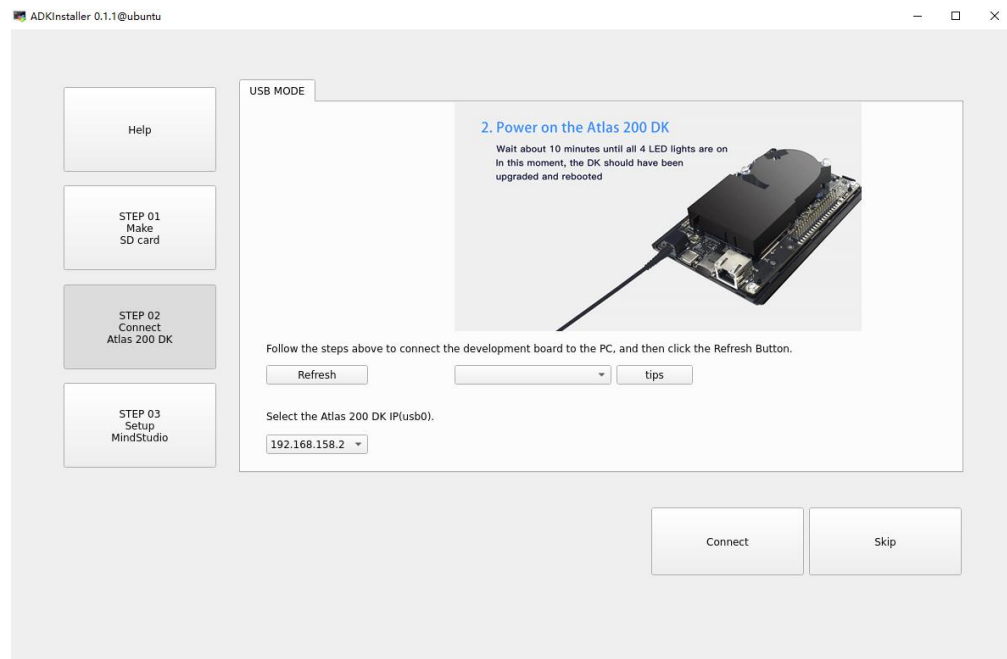
Figure 4-10 Card made successfully



Step 2 Connect to the developer board.

1. Insert the SD card into the Atlas 200 DK.
2. Use Type-C cables to connect the PC, USB port, and Type-C port on the Atlas 200 DK.
3. Power on the Atlas 200 DK. About 15 minutes later, if the four indicators on the board are all on, the board is started properly.
4. Choose **STEP 02 Connect Atlas 200 DK**.

Figure 4-11 Connecting to the Atlas 200 DK



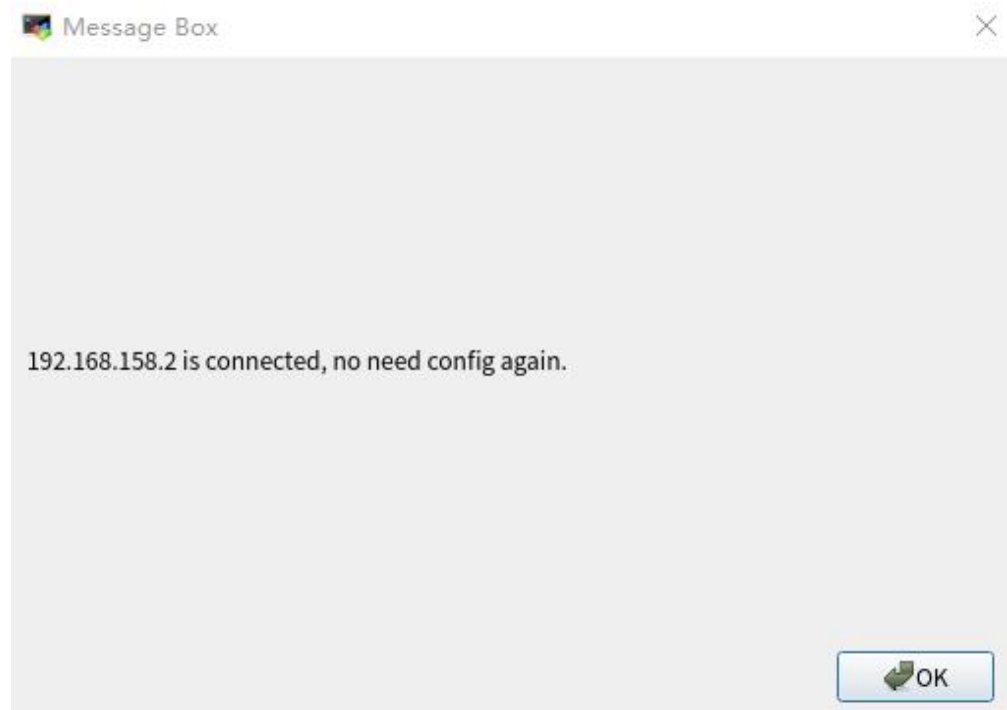
5. Click **Refresh**. The virtual NIC is automatically selected. If there are multiple developer boards, select the corresponding virtual NICs.

Select the IP address of the Atlas 200 DK. Click **Connect** to connect to the board.

If you have any questions about the developer board connection, click **tips** to view the connection details.

If the page shown in [Figure 4-12](#) is displayed, the developer board is successfully connected.

Figure 4-12 Connected successfully



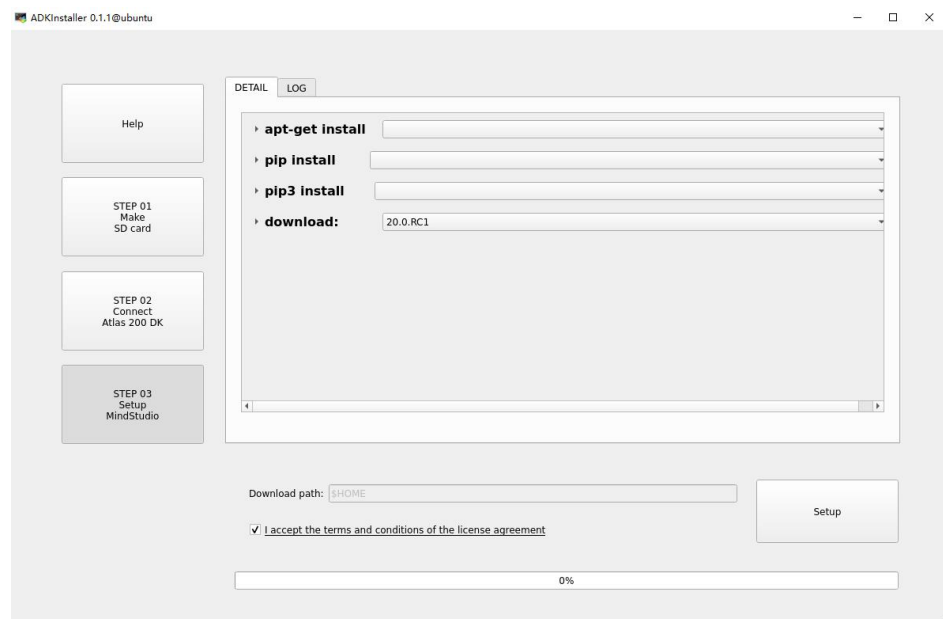
6. Click **OK** to complete the Atlas 200 DK configuration.

----End

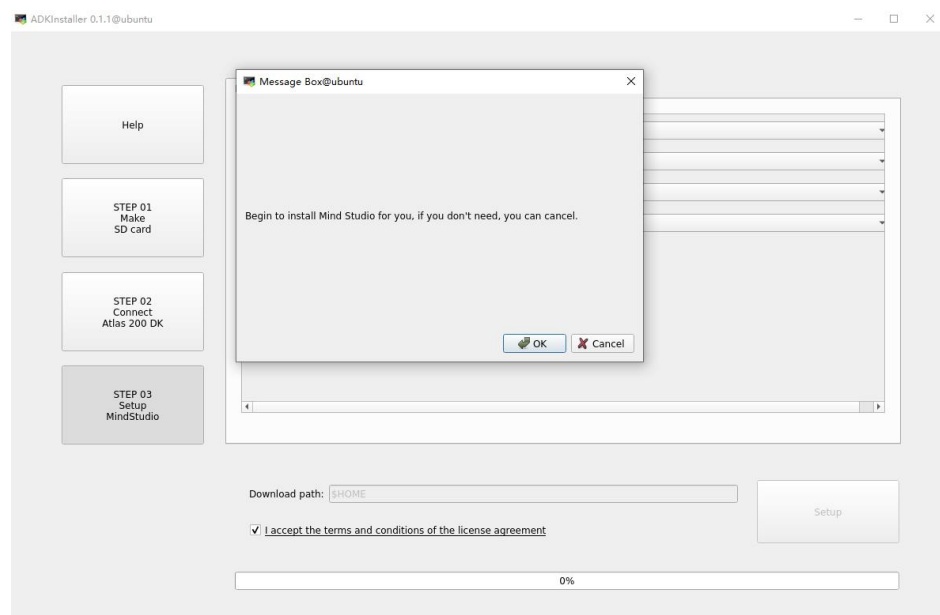
Installing the Development Environment

Step 1 Install MindStudio.

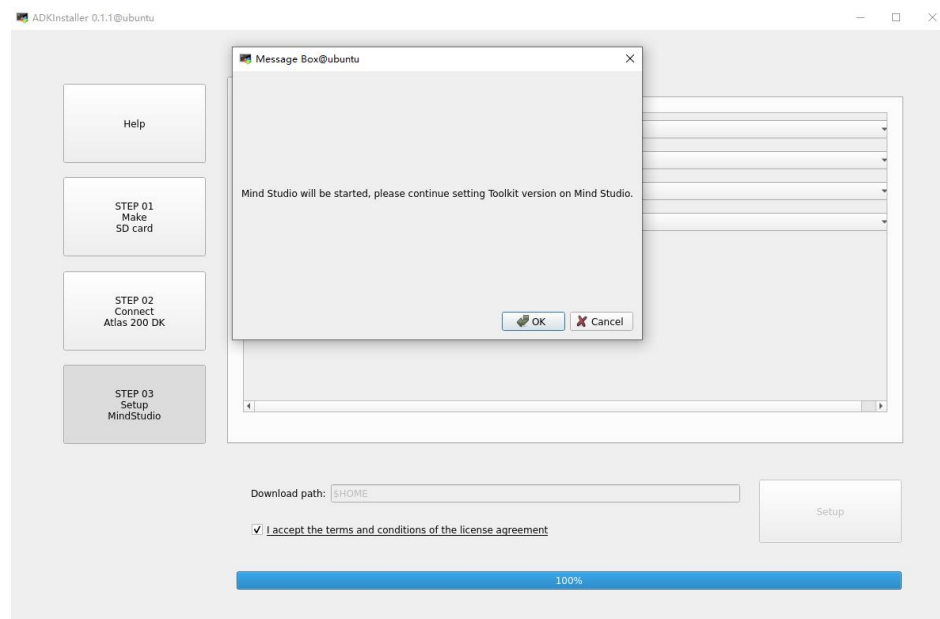
1. Choose **STEP 03 Setup MindStudio**. The MindStudio installation page is displayed.
2. As shown in [Figure 4-13](#), after setting the following parameters, click **Setup** in the lower right corner. The tool automatically downloads and installs the environment dependencies.
 - Select **I accept the terms and conditions of the license agreement**.
 - **apt-get install**: Ubuntu installation dependency
 - **pip install**: Python 2 installation dependency
 - **pip3 install**: Python 3 installation dependency
 - **download**: Version of the software package to be downloaded
 - **Download Path**: Path for downloading the dependencies

Figure 4-13 Preparing for MindStudio installation

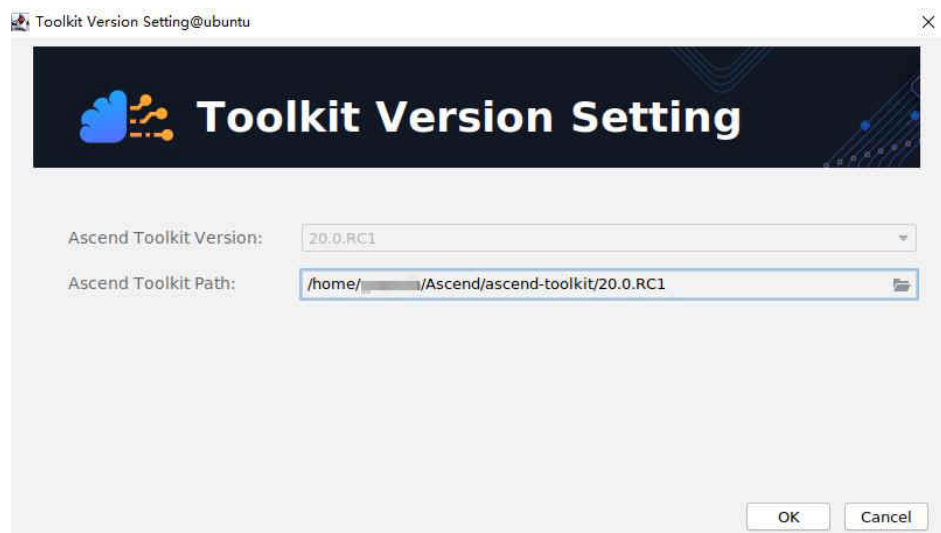
3. After the dependencies are installed, a message is displayed, indicating that MindStudio starts to be installed, as shown in [Figure 4-14](#).

Figure 4-14 MindStudio installation

4. Select **OK** to start installing MindStudio.
After the installation is complete, the information shown in [Figure 4-15](#) is displayed. Click **OK** to start MindStudio.

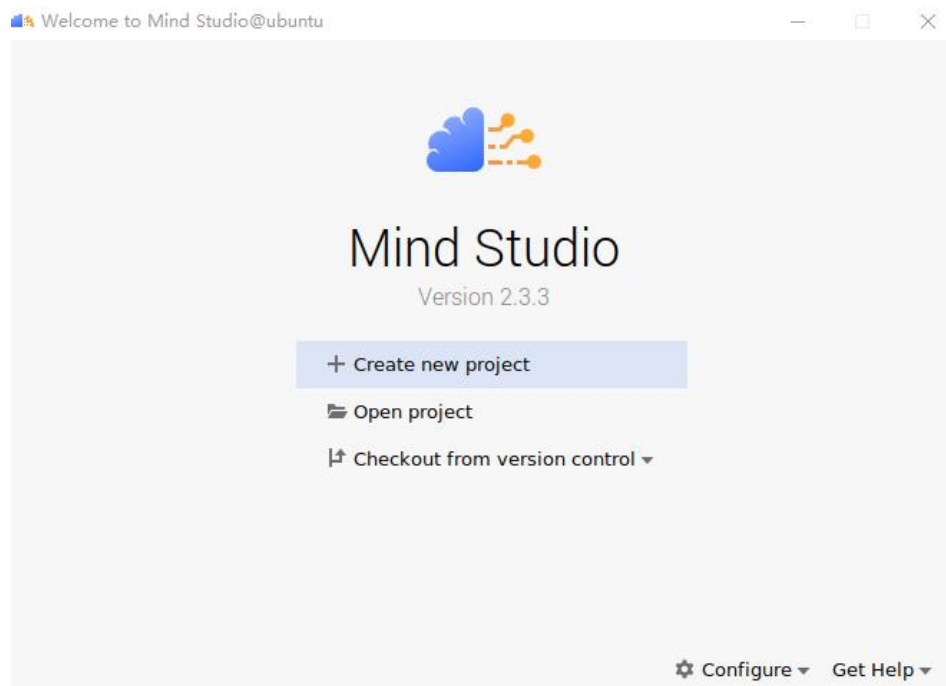
Figure 4-15 Installing MindStudio

5. When MindStudio is started, a dialog box shown in [Figure 4-16](#) is displayed. Click **OK** to complete the configuration.
 - **Ascend Toolkit Path:** Select the path of the development kit installed by the MindStudio installation user.

Figure 4-16 Importing configuration

6. Go to the `${HOME}/MindStudio-ubuntu/bin` directory where the MindStudio startup script is stored and run the following command to start MindStudio, as shown in [Figure 4-17](#).
./MindStudio.sh

Figure 4-17 Starting MindStudio



Step 2 Perform follow-up operations.

Use MindStudio to develop applications, convert models, and develop custom operator. For details, see the [MindStudio User Guide](#).

-----End

5 Common Operations

- [5.1 Upgrade and Uninstallation](#)
- [5.2 Powering on Atlas 200 DK](#)
- [5.3 Powering off the Atlas 200 DK Developer Board](#)
- [5.4 Connecting the Atlas 200 DK over a Serial Port](#)
- [5.5 Checking the Software Versions of Atlas 200 DK](#)
- [5.6 Checking the Version of the Motherboard of the Developer Board](#)
- [5.7 Checking the Version of the Atlas 200 AI Accelerator Module](#)
- [5.8 Checking the Firmware Version of the Developer Board](#)
- [5.9 Viewing the Channel to Which a Camera Belongs](#)
- [5.10 Installing the Windows USB Network Adapter Driver](#)
- [5.11 Setting User Account Validity Period](#)
- [5.12 Installing CMake 3.5.2](#)
- [5.13 Configuring a System Network Proxy](#)
- [5.14 Parameters](#)

5.1 Upgrade and Uninstallation

5.1.1 Upgrading Atlas 200 DK

This section describes how to upgrade the components of the Atlas 200 DK.

Obtaining Software Packages

[Table 5-1](#) lists the components to be upgraded.

Table 5-1 Components and software packages

Component	Software Package	Description	Procedure	How to Obtain
Driver	Ascend310-driver- <i>{software version}</i> -ubuntu18.04.aarch64-minirc.tar.gz	Driver package, including OS peripheral software, AI software stack, maintenance and testing software, as well as drivers	Upgrading a Driver	Link
Firmware	Ascend310-firmware- <i>{software version}</i> -minirc.run	Firmware package	Upgrading Firmware	Link
AI CPU	Ascend310-aicpu_kernels- <i>{software version}</i> -minirc.tar.gz	Package of AI CPU operators	Upgrading AI CPU	Link
ACLlib	Ascend-acllib- <i>{software version}</i> -ubuntu18.04.aarch64-minirc.run	Ascend Computing Language (AscendCL) library package, which is used to build and run AscendCL applications, enabling model loading and execution.	Upgrading ACLlib	Link

Upgrading a Driver

- Step 1** Copy the Driver upgrade package **Ascend310-driver-*{software version}*-ubuntu18.04.aarch64-minirc.tar.gz** to the **/opt/mini** directory on the Atlas 200 DK.

The following describes how to copy the upgrade package from the Ubuntu server (development environment) to the Atlas 200 DK.

1. Go to the directory where the Driver upgrade package is located on the Ubuntu server, log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode, and switch to the **root** user.

```
ssh HwHiAiUser@192.168.1.2
```

```
su - root
```

 NOTE

If the trust relationship fails to be established when you log in to the Atlas 200 DK in SSH mode, see [6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?](#)

2. Go to the **/opt/mini** on the Atlas 200 DK, and copy the Driver upgrade package.

cd /opt/mini

scp username@192.168.1.223:/home/ascend/software/Ascend310-driver-{software version}-ubuntu18.04.aarch64-minirc.tar.gz .

 NOTE

- *username* indicates the user name for uploading the upgrade package to the Ubuntu server.
- *192.168.1.223* indicates the IP address of the Ubuntu server that is in the same network segment as the Atlas 200 DK.
- */home/ascend/software* indicates the path on the Ubuntu server for storing the driver upgrade package.

- Step 2** Run the following command in the **/opt/mini** directory to extract the **minirc_install_phase1.sh** upgrade script from the Driver upgrade package.

tar --no-same-owner -zxf Ascend310-driver-{software version}-ubuntu18.04.aarch64-minirc.tar.gz --strip-components 2 driver/scripts/minirc_install_phase1.sh

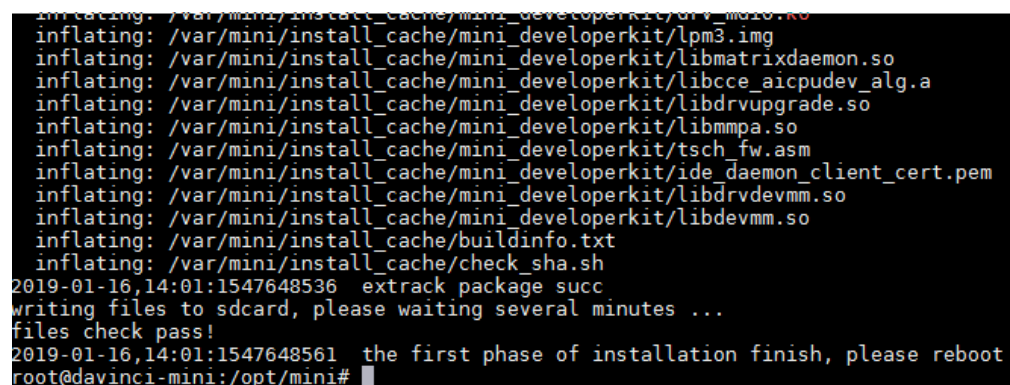
After the command is executed, the obtained **minirc_install_phase1.sh** script will replace the original script in the directory.

- Step 3** Prepare for the upgrade.

./minirc_install_phase1.sh

The information shown in [Figure 5-1](#) is displayed.

Figure 5-1 Running the upgrade script



```
inflating: /var/mini/install_cache/mini_developerkit/drv_mmpa.ro
inflating: /var/mini/install_cache/mini_developerkit/lpm3.img
inflating: /var/mini/install_cache/mini_developerkit/libmatrixdaemon.so
inflating: /var/mini/install_cache/mini_developerkit/libcce_aicpudev_alg.a
inflating: /var/mini/install_cache/mini_developerkit/libdrvupgrade.so
inflating: /var/mini/install_cache/mini_developerkit/libmmpa.so
inflating: /var/mini/install_cache/mini_developerkit/tsch_fw.asm
inflating: /var/mini/install_cache/mini_developerkit/ide_daemon_client_cert.pem
inflating: /var/mini/install_cache/mini_developerkit/libdrvdevmm.so
inflating: /var/mini/install_cache/mini_developerkit/libdevmm.so
inflating: /var/mini/install_cache/buildinfo.txt
inflating: /var/mini/install_cache/check_sha.sh
2019-01-16,14:01:1547648536  extract package succ
writing files to sdcard, please waiting several minutes ...
files check pass!
2019-01-16,14:01:1547648561  the first phase of installation finish, please reboot
root@davinci-mini:/opt/mini#
```

- Step 4** Reboot the Atlas 200 DK to complete the Driver upgrade.

reboot

NOTICE

Do not power off the Atlas 200 DK during the upgrade. The upgrade takes about 15 minutes.

----End

Upgrading Firmware

Step 1 Copy the firmware upgrade runfile **Ascend310-firmware-{software version}-minirc.run** to any directory on the Atlas 200 DK, for example, **/home/HwHiAiUser/software**.

The following describes how to copy the upgrade package from the Ubuntu server (development environment) to the Atlas 200 DK.

1. Go to the directory where the firmware upgrade runfile is located on the Ubuntu server, and log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode.

```
ssh HwHiAiUser@192.168.1.2
```

 NOTE

If the trust relationship fails to be established when you log in to the Atlas 200 DK in SSH mode, see [6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?](#)

2. Go to the path where the upgrade runfile is to be stored on the Atlas 200 DK, and copy the Firmware upgrade runfile.

```
cd /home/HwHiAiUser/software
```

```
scp username@192.168.1.223:/home/ascend/software/Ascend310-firmware-{software version}-minirc.run .
```

 NOTE

- *username* indicates the user name for uploading the upgrade package to the Ubuntu server.
- *192.168.1.223* indicates the IP address of the Ubuntu server that is in the same network segment as the Atlas 200 DK.
- */home/ascend/software* indicates the path on the Ubuntu server for storing the firmware upgrade package.

Step 2 Switch to the **root** user and upgrade the firmware.

```
su root
```

```
chmod +x Ascend310-firmware-{software version}-minirc.run
```

```
./Ascend310-firmware-{software version}-minirc.run --upgrade
```

After running the command, the firmware files are stored in the **/usr/local/Ascend/firmware** directory.

Step 3 Reboot the developer board to complete the firmware upgrade.

```
reboot
```

NOTICE

Do not power off the Atlas 200 DK during the upgrade. The upgrade takes about 15 minutes.

----End

Upgrading AI CPU

Step 1 Copy the AI CPU upgrade package **Ascend310-aicpu_kernels-{software version}-minirc.tar.gz** to any directory on the Atlas 200 DK. For example, **/home/HwHiAiUser/software**.

The following describes how to copy the upgrade package from the Ubuntu server (development environment) to the Atlas 200 DK.

1. Go to the directory where the AI CPU upgrade runfile is located on the Ubuntu server, and log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode.

```
ssh HwHiAiUser@192.168.1.2
```

 NOTE

If the trust relationship fails to be established when you log in to the Atlas 200 DK in SSH mode, see [6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?](#)

2. Go to the path where the upgrade package is to be stored on the Atlas 200 DK, and copy the AI CPU upgrade package.

```
cd /home/HwHiAiUser/software
```

```
scp username@192.168.1.223:/home/ascend/software/Ascend310-aicpu_kernels-{software version}-minirc.tar.gz .
```

 NOTE

- *username* indicates the user name for uploading the upgrade package to the Ubuntu server.
- *192.168.1.223* indicates the IP address of the Ubuntu server that is in the same network segment as the Atlas 200 DK.
- */home/ascend/software* indicates the path on the Ubuntu server for storing the AI CPU upgrade package.

Step 2 Decompress the upgrade package:

```
tar zxvf Ascend310-aicpu_kernels-{software version}-minirc.tar.gz
```

Step 3 Go to the directory where the AI CPU upgrade script is stored and switch to the **root** user to upgrade the AI CPU.

```
cd aicpu_kernels_device
```

```
su root
```

```
./scripts/install.sh --run
```

----End

Upgrading ACLlib

- Step 1** Copy the ACLlib upgrade runfile **Ascend-acclib-{software version}-ubuntu18.04.aarch64.run** to any directory on the Atlas 200 DK. For example, **/home/HwHiAiUser/software**.

The following describes how to copy the upgrade package from the Ubuntu server (development environment) to the Atlas 200 DK.

1. Go to the directory where the ACLlib upgrade runfile is located on the Ubuntu server, and log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode.

```
ssh HwHiAiUser@192.168.1.2
```

 NOTE

If the trust relationship fails to be established when you log in to the Atlas 200 DK in SSH mode, see [6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?](#)

2. Go to the path where the upgrade runfile is to be stored on the Atlas 200 DK, and copy the ACLlib upgrade runfile.

```
cd /home/HwHiAiUser/software
```

```
scp username@192.168.1.223:/home/ascend/software/Ascend-acclib-  
{software version}-ubuntu18.04.aarch64-minirc.run .
```

 NOTE

- *username* indicates the user name for uploading the upgrade package to the Ubuntu server.
- *192.168.1.223* indicates the IP address of the Ubuntu server that is in the same network segment as the Atlas 200 DK.
- */home/ascend/software* indicates the path on the Ubuntu server for storing the ACLlib upgrade package.

- Step 2** Upgrade the ACLlib:

```
chmod +x Ascend-acclib-{software version}-ubuntu18.04.aarch64-minirc.run  
./Ascend-acclib-{software version}-ubuntu18.04.aarch64-minirc.run --upgrade  
----End
```

After the upgrade is complete, you can view the version number of each component by referring to [5.5 Checking the Software Versions of Atlas 200 DK](#).

5.1.2 Upgrading the Development Kit

This section describes how to upgrade the development kit (**Ascend-Toolkit-{version}-{aarch}-linux_{gccversion}.run**). After the software package is upgraded, services are not affected.

If the following information is displayed during the upgrade of the development kit in the x86 system, asking you whether to perform a hot reset, enter **n**. After the upgrade is complete, restart the OS for the setting to take effect. In the current version, only **n** is supported.

The installation of aicpu_kernels needs to restart the device to take effect, do you want to hot_reset the device? [y/n]

Step 1 Log in to the installation environment as the installation user of the software package.

Step 2 Go to the directory where the software packages are stored.

Step 3 Grant the execute permission on the software package.

```
chmod +x software package name.run
```

Step 4 Run the following command to check the consistency and integrity of the software package installation files:

```
./software package name.run --check
```

Step 5 Upgrade the software.

- If you specify the path when installing the software package, run the following command:

```
./software package name.run --upgrade --install-path=<path>
```

In the preceding command, *<path>* indicates the specified installation directory of the software package. Replace the software package name with the actual one.

- If you do not specify a path when installing the software package, run the following command:

```
./software package name.run --upgrade
```

----End

5.1.3 Uninstalling the Development Kit

This section describes how to uninstall the development kit (**Ascend-Toolkit-*{version}*-*{aarch}*-linux_*{gccversion}*.run**).

Step 1 Log in to the installation environment as the installation user of the software package.

Step 2 Go to the directory where the software packages are stored.

Step 3 Uninstall the software package.

- If you specify a path when installing a software package, run the **./software package name.run --uninstall --install-path=<path>** to uninstall the software package.

<path> indicates the specified software package installation path. Replace the software package name with the actual package name.

- If you do not specify a path when installing the software package, run the **./software package name.run --uninstall** command to uninstall the software package.

If the following information is displayed, the software is successfully uninstalled:

```
[INFO] xxx uninstall success  
[INFO] process end
```

xxx indicates the name of the software package to be uninstalled.

----End

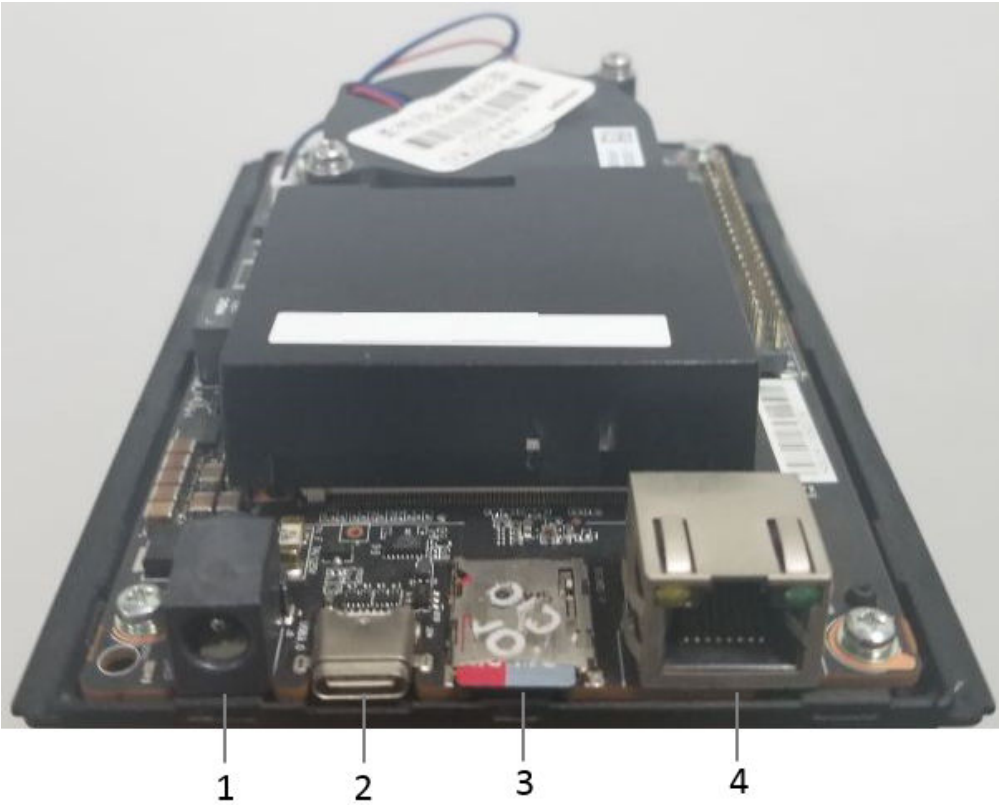
5.2 Powering on Atlas 200 DK

NOTICE

Do not power down the Atlas 200 DK during the first boot or an upgrade. Otherwise, the Atlas 200 DK may be damaged. Wait at least 2s after it is powered down before powering it on again.

- Step 1** Connect the power module to the external power supply. [Figure 5-2](#) shows the power port on the Atlas 200 DK. After connected to the power supply, the Atlas 200 DK automatically boots.

Figure 5-2 Port description



1	Power port
2	USB
3	SD card
4	Network port

- Step 2** Check the status of the indicator to ensure that the Atlas 200 DK is powered on properly.

The indicators are visible only after the top cover is removed. The following shows the positions of the indicators.

Figure 5-3 LED positions when the mainboard is IT21DMDA

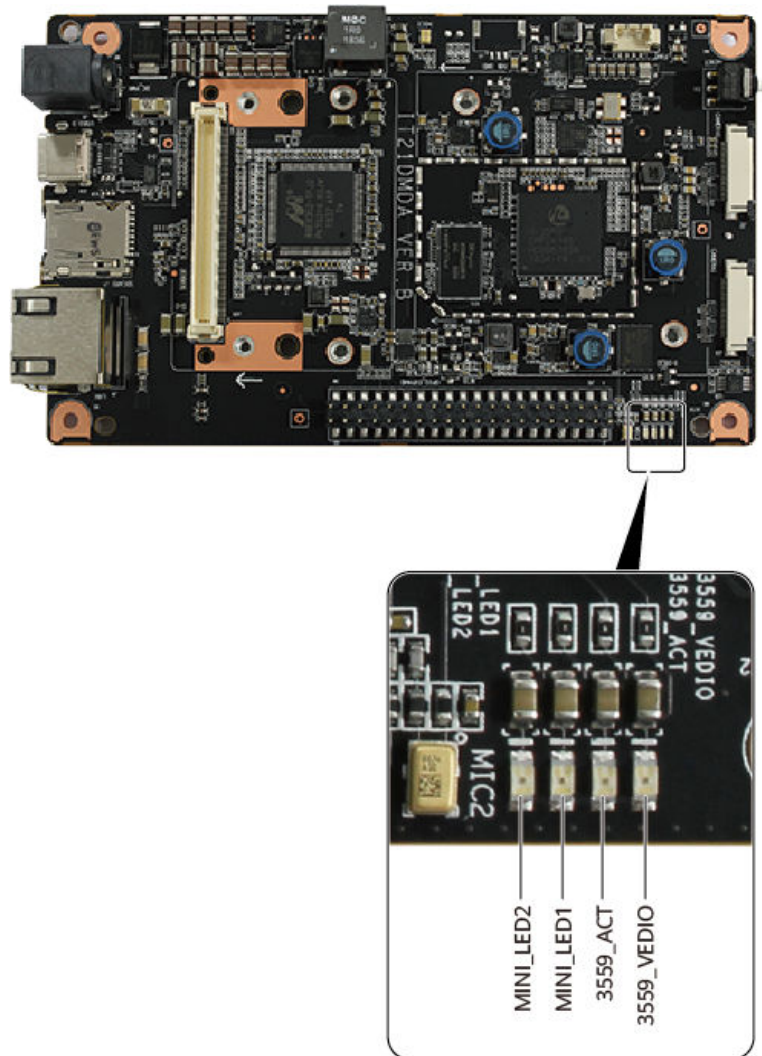
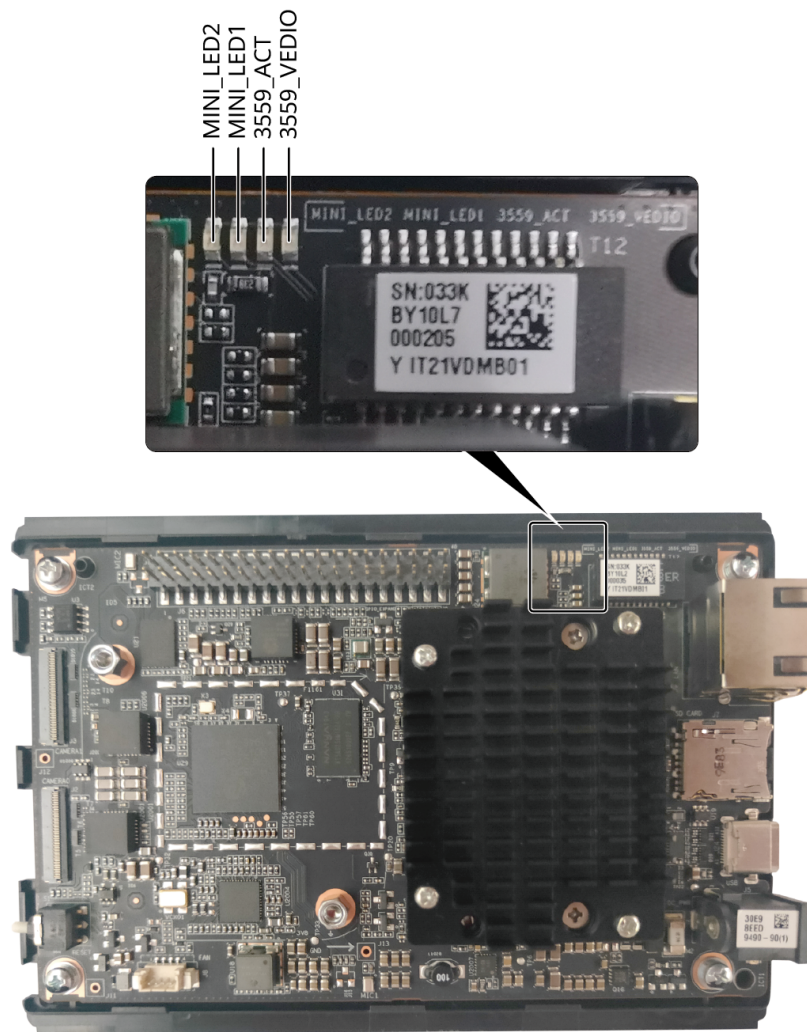


Figure 5-4 LED positions when the mainboard is IT21VDMB



NOTE

The silkscreen of the indicators is **MINI_LED2**, **MINI_LED1**, **3559_ACT**, and **3559_VEDIO** from left to right, corresponding to the indicators from left to right.

Table 5-2 Indicator status 1

MINI_LE D2	MINI_LE D1	Developer Board Status of the Atlas 200 DK developer kit (model 3000)	Important Notes
Off	Off	The developer board of the Atlas 200 DK developer kit (model 3000) is started.	You can power off or restart the developer board of the Atlas 200 DK developer kit (model 3000).

MINI_LE D2	MINI_LE D1	Developer Board Status of the Atlas 200 DK developer kit (model 3000)	Important Notes
Off	On	Ascend 310 is started.	You can power off or restart the developer board of the Atlas 200 DK developer kit (model 3000) except during version upgrade.
Blink	Blink	The firmware is being upgraded.	- Do not power off or restart the developer board of the Atlas 200 DK developer kit (model 3000). Otherwise, the firmware upgrade may be incomplete and the board may be damaged. - The firmware upgrade process is a part of the version upgrade. The upgrade takes about 15 minutes.
On	On	The developer board of the Atlas 200 DK developer kit (model 3000) is started.	You can power off or restart the developer board of the Atlas 200 DK developer kit (model 3000).

Table 5-3 Indicator status 2

3559_A CT	3559_V EDIO	Developer Board Status of the Atlas 200 DK developer kit (model 3000)	Important Notes
Off	Off	The Hi3559C system is not started.	None
Off	On	The Hi3559C system is being started.	None
On	On	The startup process of the Hi3559C system is complete.	None

----End

5.3 Powering off the Atlas 200 DK Developer Board

Precautions

Determine whether the Atlas 200 DK developer board can be powered off based on the description in [Step 2](#).

Procedure

- Step 1** Disconnect the power cable from the power port to power off the Atlas 200 DK developer board.

NOTICE

The Atlas 200 DK cannot be shut down by using the OS-level shutdown command.

----End

5.4 Connecting the Atlas 200 DK over a Serial Port

Connecting to the Atlas 200 AI Accelerator Module over a Serial Port

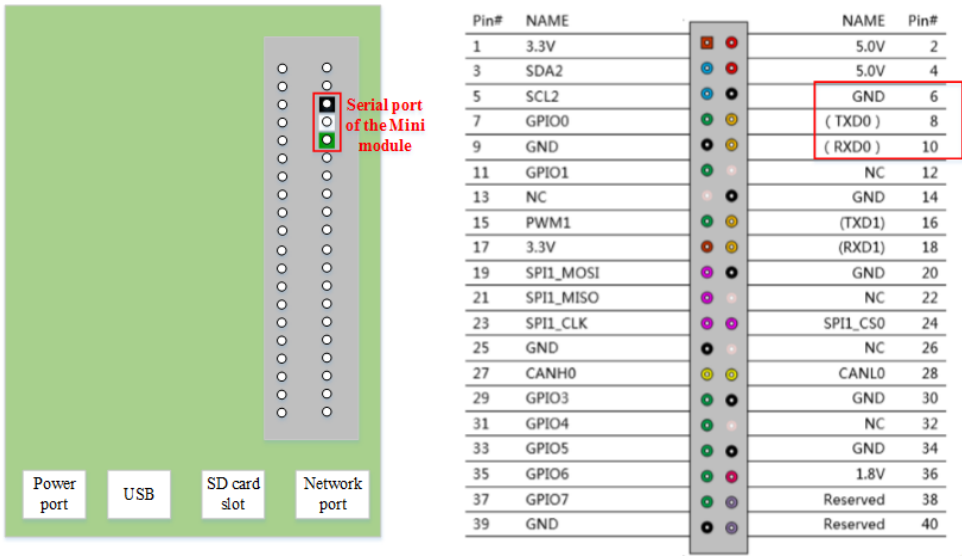
You can view the boot information about the AI accelerator module on the Atlas 200 DK over a serial port.

 NOTE

This serial port is used only for viewing boot information. After the Atlas 200 AI accelerator module is started, its serial port is disabled, and the Atlas 200 AI accelerator module cannot be logged in to.

[Figure 5-5](#) shows how to connect the Atlas 200 AI accelerator module by using a serial cable.

Figure 5-5 Serial port connection of the Atlas 200 AI accelerator module



Serial port on the Atlas 200 AI accelerator module: Connect a cable to the serial port according to the colors specified in [Figure 5-5](#).

Requirements for the serial cable: USB-to-serial cable (3.3 V)

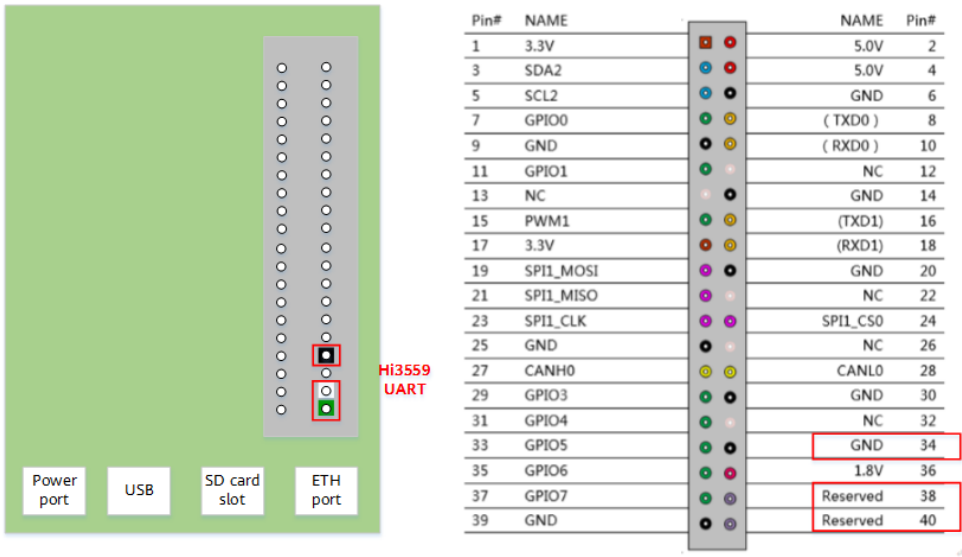
Connecting to the Hi3559 Module over a Serial Port

The Atlas 200 DK provides a serial port for connecting to the Hi3559 module. [Figure 5-6](#) shows the serial port connection diagram.

NOTE

This serial port is used only for viewing boot information. After the Hi3559 module is started, its serial port is disabled, and the Hi3559 module cannot be logged in to.

Figure 5-6 Hi3559 serial port connection



Connect the serial cable to the Hi3559 serial port according to the colors specified in [Figure 5-6](#).

Requirements for the serial cable: USB-to-serial cable (3.3 V)

5.5 Checking the Software Versions of Atlas 200 DK

This section describes how to check the software versions of the Atlas 200 DK.

Step 1 Log in to the Atlas 200 DK as the **HwHiAiUser** user in SSH mode from the Ubuntu server.

```
ssh HwHiAiUser@192.168.1.2
```

NOTE

Replace **192.168.1.2** with the actual IP address of the Atlas 200 DK.

Step 2 Switch to the **root** user, and check the Driver version:

```
su root  
cat /var/davinci/driver/version.info
```

Step 3 Check the Firmware version and the versions of the valid components:

1. Switch to the **root** user.

```
su root
```

2. Check the Firmware version:

```
cd /var/davinci/driver/  
./upgrade-tool --device_index -1 --system_version
```

If the Firmware component of the Atlas 200 DK has been upgraded, you can run the following command to check the version number:

```
cat /usr/local/Ascend/firmware/version.info
```

3. Check the Firmware version:

```
cd /var/davinci/driver/  
./upgrade-tool --device_index -1 --component -1 --version
```

Step 4 Check the AI CPU version:

1. Switch to the **root** user.

```
su root
```

2. Check the AI CPU version:

```
cat /var/davinci/aicpu_kernels/version.info
```

Step 5 Check the ACLlib version:

```
cat /home/HwHiAiUser/Ascend/acclib/version.info
```

----End

5.6 Checking the Version of the Motherboard of the Developer Board

You can obtain the version of the motherboard by checking the PCB version of the developer board.

Step 1 Create a code file for querying the version number of the developer board.

Create a **i2c_tool_atlas200dk.c** file in any directory of the Ubuntu server as a common user.

touch i2c_tool_atlas200dk.c

Copy the following code to the **i2c_tool_atlas200dk.c** file:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/select.h>
#include <sys/time.h>
#include <errno.h>
#include <string.h>
#define I2C0_DEV_NAME "/dev/i2c-0"
#define I2C1_DEV_NAME "/dev/i2c-1"
#define I2C2_DEV_NAME "/dev/i2c-2"
#define I2C3_DEV_NAME "/dev/i2c-3"
#define I2C_RETRIES 0x0701
#define I2C_TIMEOUT 0x0702
#define I2C_SLAVE 0x21
#define I2C_RDWR 0x0707
#define I2C_BUS_MODE 0x0780
#define I2C_M_RD 0x01
#define PCB_ID_VER_A 0x1
#define PCB_ID_VER_B 0x2
#define PCB_ID_VER_C 0x3
#define PCB_ID_VER_D 0x4
#define I2C_SLAVE_PCA9555_BOARDINFO (0x20)
#define BOARD_ID_DEVELOP_C (0xCE)
#define DEVELOP_A_BOM_PCB_MASK (0xF)
#define DEVELOP_C_BOM_PCB_MASK (0x7)

typedef unsigned char uint8;
typedef unsigned short uint16;
struct i2c_msg
{
    uint16 addr; /* slave address */
    uint16 flags;
    uint16 len;
    uint8 *buf; /*message data pointer*/
};
struct i2c_rdwr_ioctl_data
{
    struct i2c_msg *msgs; /*i2c_msg[] pointer*/
    int nmsgs; /*i2c_msg Num*/
};
static uint8 i2c_init(char *i2cdev_name);
static uint8 i2c_read(uint8 slave, unsigned char reg, unsigned char *buf);
int fd = 0;
```

```
static uint8 i2c_read(unsigned char slave, unsigned char reg, unsigned char *buf)
{
    int ret;
    struct i2c_rdwr_ioctl_data ssm_msg;
    unsigned char regs[2] = {0};

    regs[0] = reg;
    regs[1] = reg;
    ssm_msg.nmsgs=2;
    ssm_msg.msgs=(struct i2c_msg*)malloc(ssm_msg.nmsgs*sizeof(struct i2c_msg));

    if(!ssm_msg.msgs)
    {
        printf("Memory alloc error!\n");
        return -1;
    }
    (ssm_msg.msgs[0]).flags=0;
    (ssm_msg.msgs[0]).addr=slave;
    (ssm_msg.msgs[0]).buf= regs;
    (ssm_msg.msgs[0]).len=1;

    (ssm_msg.msgs[1]).flags=I2C_M_RD;
    (ssm_msg.msgs[1]).addr=slave;
    (ssm_msg.msgs[1]).buf=buf;
    (ssm_msg.msgs[1]).len=2;

    ret=ioctl(fd, I2C_RDWR, &ssm_msg);
    if(ret<0)
    {
        printf("read data error,ret=%#x, errno=%#x, %s!\n",ret, errno, strerror(errno));
        free(ssm_msg.msgs);
        return -1;
    }

    free(ssm_msg.msgs);
    return 0;
}

static uint8 i2c_init(char *i2cdev_name)
{
    fd = open(i2cdev_name, O_RDWR);
    if(fd < 0)
    {
        printf("Can't open %s!\n", i2cdev_name);
        return -1;
    }

    if(ioctl(fd, I2C_RETRIES, 1)<0)
    {
        printf("set i2c retry fail!\n");
        return -1;
    }

    if(ioctl(fd, I2C_TIMEOUT, 1)<0)
    {
        printf("set i2c timeout fail!\n");
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[])
{
    char *dev_name = I2C0_DEV_NAME;
    uint8 board_id;
    uint8 pcb_id;
    uint8 buff[2] = {0};
    uint8 ret;
```

```
if (i2c_init(dev_name))
{
    printf("i2c init fail!\n");
    close(fd);
    return -1;
}
usleep(1000*100);

ret = i2c_read(I2C_SLAVE_PCA9555_BOARDINFO, 0x0, buff);
if (ret != 0)
{
    printf("read %s %#x fail, ret %d\n", dev_name, I2C_SLAVE_PCA9555_BOARDINFO, ret);
}

close(fd);

board_id = buff[0];
if (board_id == BOARD_ID_DEVELOP_C)
{
    pcb_id = (buff[1]>>3)&DEVELOP_C_BOM_PCB_MASK;
}
else
{
    pcb_id = (buff[1]>>4)&DEVELOP_A_BOM_PCB_MASK;
}

// show PCB ID;
switch (pcb_id)
{
case PCB_ID_VER_A:
    printf("PCB version is: Ver.A !\n");
    break;
case PCB_ID_VER_B:
    printf("PCB version is: Ver.B !\n");
    break;
case PCB_ID_VER_C:
    printf("PCB version is: Ver.C !\n");
    break;
case PCB_ID_VER_D:
    printf("PCB version is: Ver.D !\n");
    break;
default:
    break;
}
return 0;
}
```

Step 2 Compile the file to obtain an executable file for obtaining the PCB version number.

Run the following command to compile the **i2c_tool_atlas200dk.c** file into a file that can be executed on the developer board:

```
aarch64-linux-gnu-gcc i2c_tool_atlas200dk.c -o atlas200dk_version_tool
```

atlas200dk_version_tool indicates the name of the executable file.

Step 3 Upload the executable file generated in [Step 2](#) to the developer board.

For example, upload the file to the home directory of the **HwHiAiUser** user of the developer board.

```
scp atlas200dk_version_tool HwHiAiUser@192.168.1.2:/home/HwHiAiUser
```

Step 4 Log in to the developer board as the **HwHiAiUser** user in SSH mode and query the PCB version number of the developer board.

```
ssh HwHiAiUser@192.168.1.2
```

Switch to the **root** user and execute the query script.

su root

./atlas200dk version tool

The following information is displayed, indicating that the developer board is a VB version.

```
root@davinci-mini:/home/HwHiAiUser# ./atlas200dk_version_tool
PCB version is: Ver.B !
```

----End

5.7 Checking the Version of the Atlas 200 AI Accelerator Module

You can determine the version of the Atlas 200 AI acceleration module based on the value of **boardid** or the PCB version of the Atlas 200 AI acceleration module. The following describes two query methods.

Method 1 (CLI Mode)

Step 1 Log in to the Atlas 200 DK developer board as the **HwHiAiUser** user in SSH mode.

Step 2 Run the following command to check the version of the Atlas 200 AI accelerator module:

```
cat /proc/cmdline
```

[illegible]

Determine the version of the Atlas 200 AI accelerator module based on the value of **boardid**.

- **boardid = 000:** Indicates that the Atlas 200 AI accelerator module is a VC or earlier version.
- **boardid = 004:** Indicates that the Atlas 200 AI accelerator module is a VD version.

---End

Method 2 (Checking the PCB Version of the Atlas 200 AI Acceleration Module)

Step 1 Create a code file for querying the version number of the developer board.

Create a **i2c_tool_mini.c** file in any directory as a common user of the Ubuntu server.

touch i2c tool mini.c

Copy the following code to the **i2c_tool_mini.c** file:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/select.h>
#include <sys/time.h>
#include <errno.h>
#include <string.h>
#define I2C0_DEV_NAME "/dev/i2c-0"
#define I2C1_DEV_NAME "/dev/i2c-1"
#define I2C2_DEV_NAME "/dev/i2c-2"
#define I2C3_DEV_NAME "/dev/i2c-3"
#define I2C_RETRIES 0x0701
#define I2C_TIMEOUT 0x0702
#define I2C_SLAVE 0x21
#define I2C_RDWR 0x0707
#define I2C_BUS_MODE 0x0780
#define I2C_M_RD 0x01
#define PCB_ID_VER_A 0x10
#define PCB_ID_VER_B 0x20
#define PCB_ID_VER_C 0x30
#define PCB_ID_VER_D 0x40
typedef unsigned char uint8;
typedef unsigned short uint16;
struct i2c_msg
{
    uint16 addr; /* slave address */
    uint16 flags;
    uint16 len;
    uint8 *buf; /*message data pointer*/
};
struct i2c_rdwr_ioctl_data
{
    struct i2c_msg *msgs; /*i2c_msg[] pointer*/
    int nmsgs; /*i2c_msg Nums*/
};
static uint8 i2c_init(char *i2cdev_name);
static uint8 i2c_write(uint8 slave, unsigned char reg, unsigned char value);
static uint8 i2c_read(uint8 slave, unsigned char reg, unsigned char *buf);
int fd = 0;
static uint8 i2c_write(uint8 slave, unsigned char reg, unsigned char value)
{
    int ret;
    struct i2c_rdwr_ioctl_data ssm_msg;
    unsigned char buf[2]={0};

    ssm_msg.nmsgs=1;
    ssm_msg.msgs=(struct i2c_msg*)malloc(ssm_msg.nmsgs*sizeof(struct i2c_msg));
    if(!ssm_msg.msgs)
    {
        printf("Memory alloc error!\n");
        return -1;
    }

    buf[0] = reg;
    buf[1] = value;

    (ssm_msg.msgs[0]).flags=0;
    (ssm_msg.msgs[0]).addr=(uint16)slave;
    (ssm_msg.msgs[0]).buf=buf;
    (ssm_msg.msgs[0]).len=2;

    ret=ioctl(fd, I2C_RDWR, &ssm_msg);
    if(ret<0)
    {
        printf("write error, ret=%#x, errno=%#x, %s!\n",ret, errno, strerror(errno));
```

```
        free(ssm_msg.msgs);
        return -1;
    }

    free(ssm_msg.msgs);
    return 0;
}

static uint8 i2c_read(unsigned char slave, unsigned char reg, unsigned char *buf)
{
    int ret;
    struct i2c_rdwr_ioctl_data ssm_msg;
    unsigned char regs[2] = {0};

    regs[0] = reg;
    regs[1] = reg;
    ssm_msg.nmsgs=2;
    ssm_msg.msgs=(struct i2c_msg*)malloc(ssm_msg.nmsgs*sizeof(struct i2c_msg));

    if(!ssm_msg.msgs)
    {
        printf("Memory alloc error!\n");
        return -1;
    }
    (ssm_msg.msgs[0]).flags=0;
    (ssm_msg.msgs[0]).addr=slave;
    (ssm_msg.msgs[0]).buf= regs;
    (ssm_msg.msgs[0]).len=1;

    (ssm_msg.msgs[1]).flags=I2C_M_RD;
    (ssm_msg.msgs[1]).addr=slave;
    (ssm_msg.msgs[1]).buf=buf;
    (ssm_msg.msgs[1]).len=1;

    ret=ioctl(fd, I2C_RDWR, &ssm_msg);
    if(ret<0)
    {
        printf("read data error,ret=%#x, errno=%#x, %s!\n",ret, errno, strerror(errno));
        free(ssm_msg.msgs);
        return -1;
    }

    free(ssm_msg.msgs);
    return 0;
}

static uint8 i2c_init(char *i2cdev_name)
{
    fd = open(i2cdev_name, O_RDWR);
    if(fd < 0)
    {
        printf("Can't open %s!\n", i2cdev_name);
        return -1;
    }

    if(ioctl(fd, I2C_RETRIES, 1)<0)
    {
        printf("set i2c retry fail!\n");
        return -1;
    }

    if(ioctl(fd, I2C_TIMEOUT, 1)<0)
    {
        printf("set i2c timeout fail!\n");
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[])
{
    char *dev_name = I2C0_DEV_NAME;
```

```
uint8 slave;
uint8 reg;
uint8 data;
int ret;

if (i2c_init(dev_name))
{
    printf("i2c init fail!\n");
    close(fd);
    return -1;
}
usleep(1000*100);

// Read PCB ID
slave = I2C_SLAVE;
reg = 0x07;
data = 0x5A;

ret = i2c_read(slave, reg, &data);
if (ret != 0)
{
    printf("read %s %#x %#x to %#x fail!\n", dev_name, slave, data, reg);
}

slave = I2C_SLAVE;
reg = 0x07;
data = data|0xF0;

ret = i2c_write(slave, reg, data);
if (ret != 0)
{
    printf("write %s %#x %#x to %#x fail!\n", dev_name, slave, data, reg);
}

slave = I2C_SLAVE;
reg = 0x01;
data = 0x5A;

ret = i2c_read(slave, reg, &data);
if (ret != 0)
{
    printf("read %s %#x %#x to %#x fail!\n", dev_name, slave, data, reg);
}

close(fd);

// show PCB ID;
switch (data & 0xF0)
{
case PCB_ID_VER_A:
    printf("PCB version is: Ver.A !\n");
    break;
case PCB_ID_VER_B:
    printf("PCB version is: Ver.B !\n");
    break;
case PCB_ID_VER_C:
    printf("PCB version is: Ver.C !\n");
    break;
case PCB_ID_VER_D:
    printf("PCB version is: Ver.D !\n");
    break;
default:
    break;
}
return 0;
}
```

Step 2 Compile the file to obtain an executable file for obtaining the PCB version number.

Run the following command to compile the `i2c_tool_mini.c` file to a file that can be executed on the developer board:

```
aarch64-linux-gnu-gcc i2c_tool_mini.c -o mini_version_tool
```

`mini_version_tool` indicates the name of the executable file.

Step 3 Upload the executable file generated in [Step 2](#) to the developer board.

For example, upload the file to the home directory of the **HwHiAiUser** user of the developer board.

```
scp mini_version_tool HwHiAiUser@192.168.1.2:/home/HwHiAiUser
```

Step 4 Log in to the developer board as the **HwHiAiUser** user in SSH mode and query the PCB version number of the developer board.

```
ssh HwHiAiUser@192.168.1.2
```

Switch to the **root** user and execute the query script.

```
su root
```

```
./mini_version_tool
```

The following information is displayed, indicating the Atlas 200 AI accelerator module is a VC version.

```
root@davinci-mini:/home/HwHiAiUser# ./mini_version_tool
PCB version is: Ver.C !
```

----End

5.8 Checking the Firmware Version of the Developer Board

If you can log in to the OS of the Atlas 200 DK developer board, check the firmware version by referring to [5.5 Checking the Software Versions of Atlas 200 DK](#). If you cannot log in to the OS of the Atlas 200 DK developer board, check the firmware version by viewing the startup logs of the Atlas 200 DK serial port as follows:

Step 1 Connect to the AI accelerator module of the Atlas 200 DK over a serial port. For details, see [5.4 Connecting the Atlas 200 DK over a Serial Port](#).

Step 2 Power on the Atlas 200 DK developer board and view the log on the serial port terminal. The firmware version is printed, as shown in the following figure.

Figure 5-7 Querying the firmware version

```
Line 69: [NVE]current_id = 3 number of nvbin = 19, version = 268963089
Line 164: Xloader version is 00000001.00000001.00000013.00000810
Line 252: UEFI version is 1.1.13.810
```

- The Xloader version number is **1.1.T13.B810**.
- The UEFI version number is **1.1.T13.B810**.

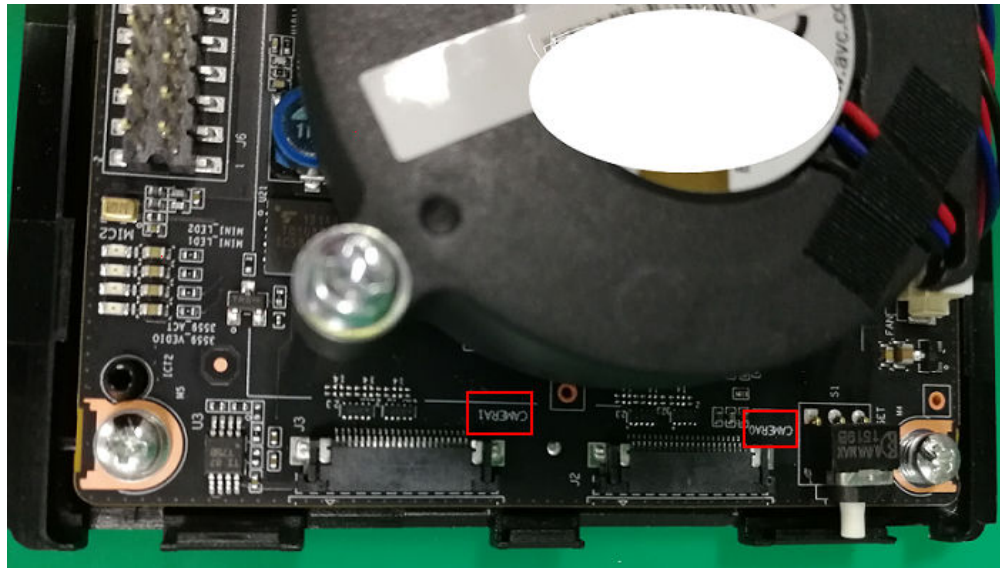
----End

5.9 Viewing the Channel to Which a Camera Belongs

The Atlas 200 DK provides two MIPI-CSI interfaces for connecting to two cameras.

You can determine the camera channel in use by viewing the developer board, as shown in [Figure 5-8](#).

Figure 5-8 Viewing the channel to which a camera belongs



- The channel corresponding to **CAMERA0** is **Channel-1**.
- The channel corresponding to **CAMERA1** is **Channel-2**.

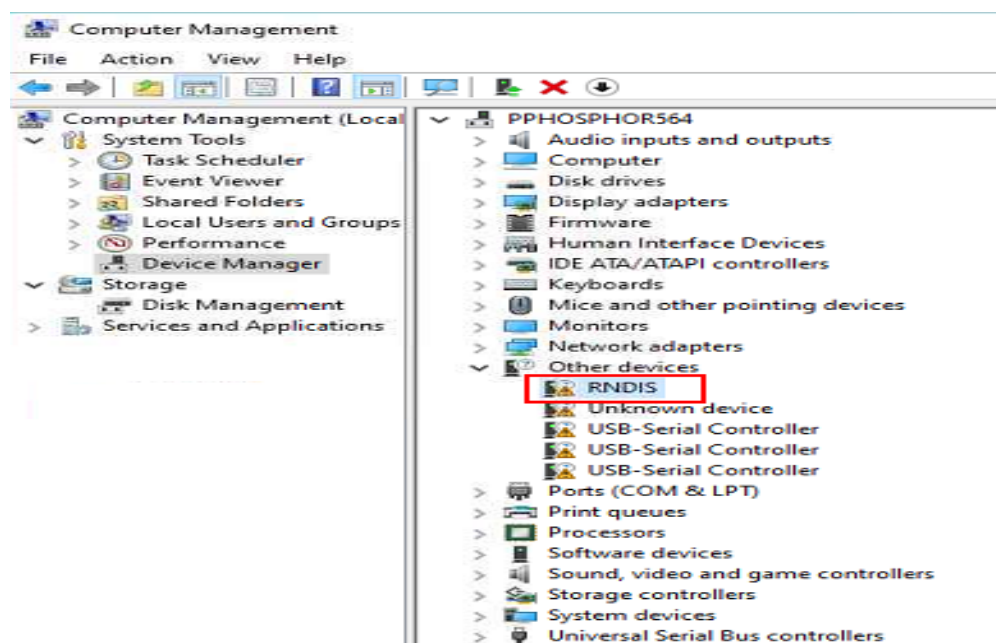
5.10 Installing the Windows USB Network Adapter Driver

If the Ubuntu server is installed through a VM running Windows, you need to install the USB NIC driver, that is, the Remote Network Driver Interface Specification (RNDIS) driver, on the Windows OS. Otherwise, when the Atlas 200 DK connects to the Windows host where Ubuntu is located through a USB cable, the USB virtual NIC of the Atlas 200 DK cannot be identified in the Ubuntu OS.

Assume that you have connected the Atlas 200 DK to the Windows host running Ubuntu using a USB cable, perform the following steps to install the RNDIS driver on Windows 10.

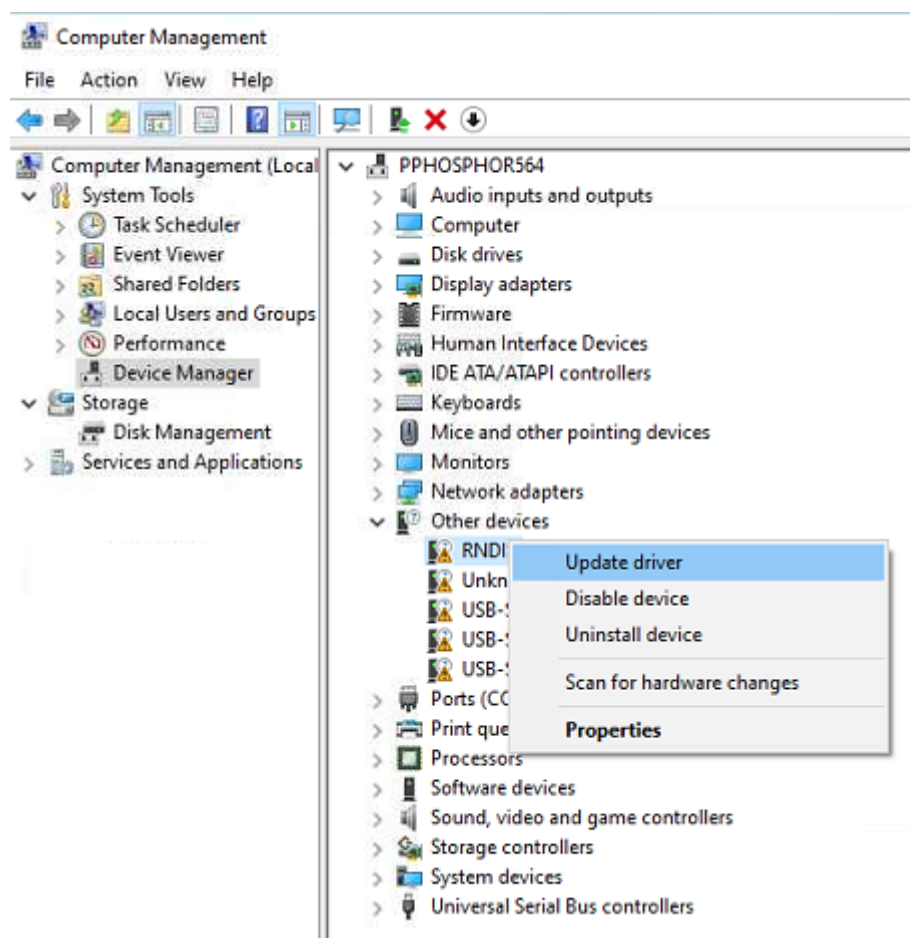
- Step 1** In the **Computer Management** window, choose **Device Manager** > **Other devices**, as shown in the following figure. **RNDIS** is in the unidentified state.

Figure 5-9 Device Manager



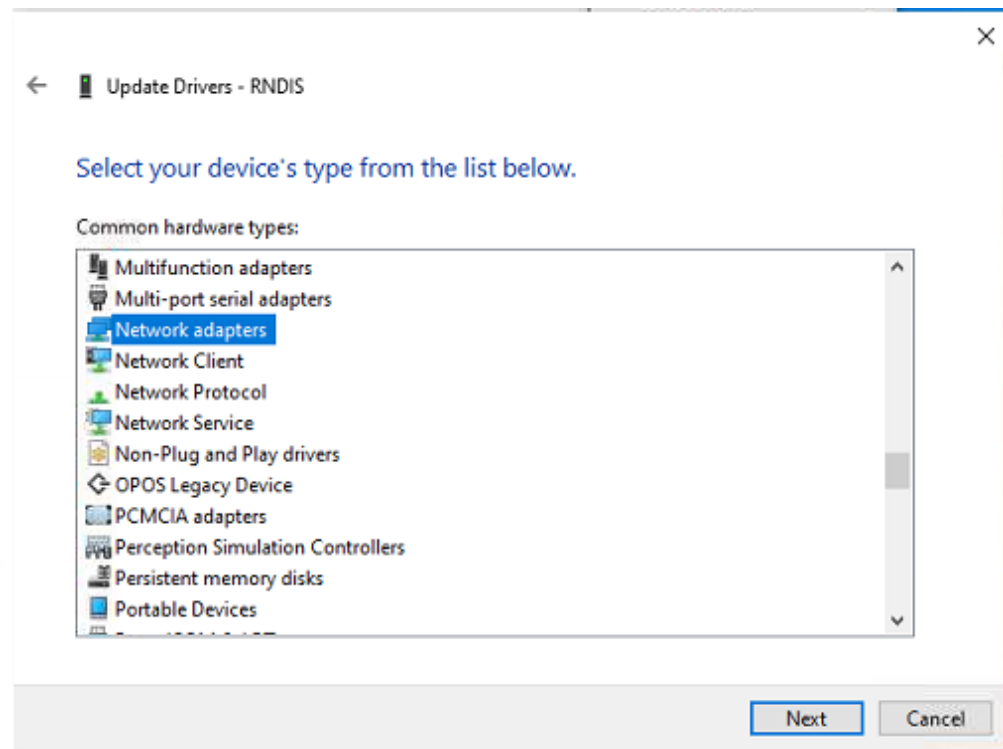
Step 2 Right-click **RNDIS** and choose **Update driver** from the shortcut menu.

Figure 5-10 Updating RNDIS



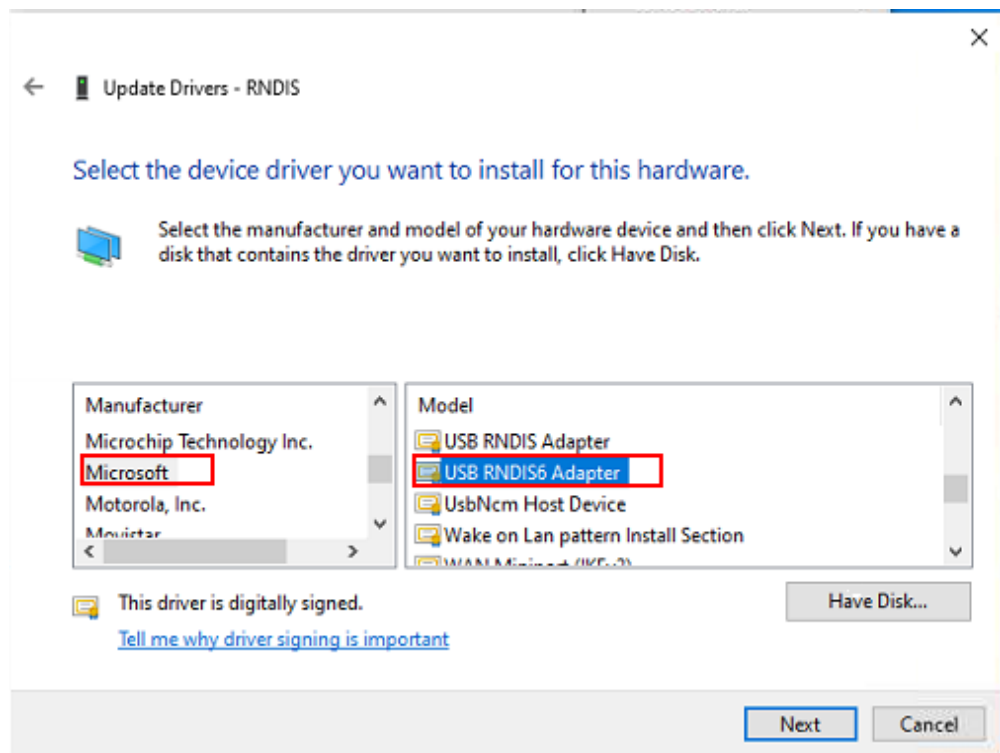
- Step 3** In the displayed **Update Drivers - RNDIS** window, click **Browse my computer for driver software**, click **Select your device's type from the list below**, and then click **Next**.
- Step 4** In the **Common hardware types** list, select **Network adapters** and click **Next**.

Figure 5-11 Selecting network adapters



- Step 5** In the **Select the device driver you want to install for this hardware** dialog box, choose **Microsoft > USB RNDIS6 Adapter**.

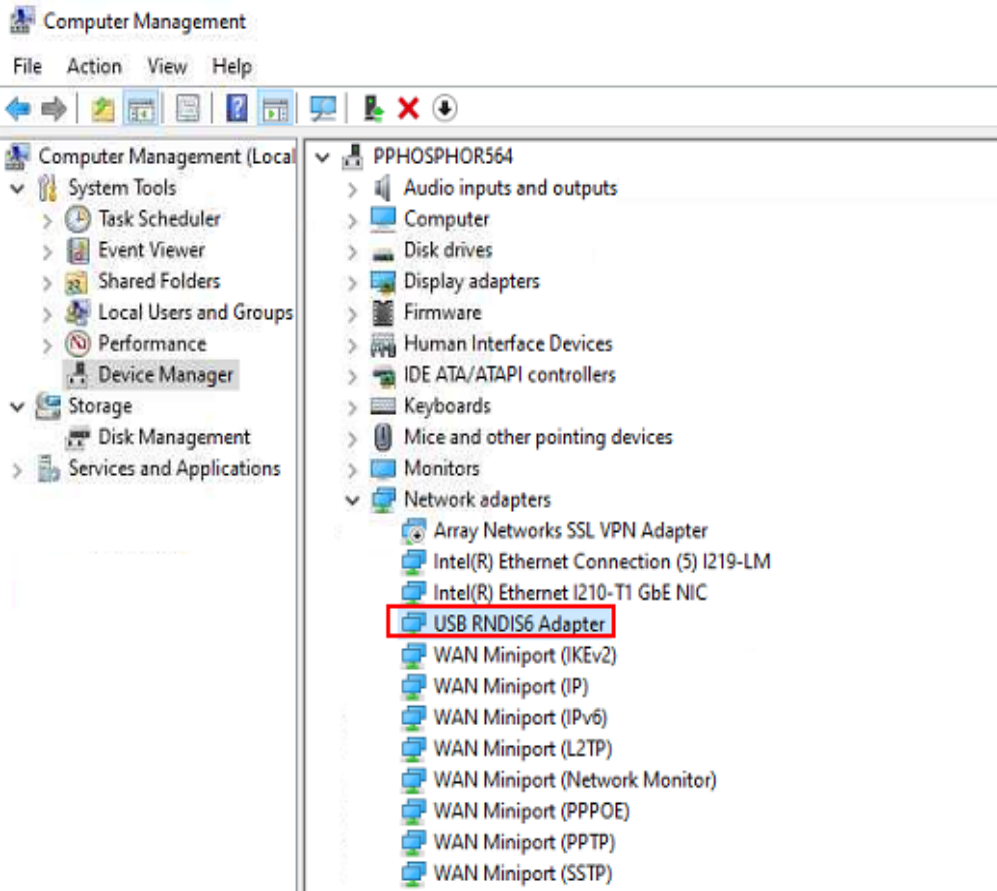
Figure 5-12 Selecting a driver



Step 6 Click **Next**. The **Update Driver Warning** dialog box is displayed. Click **Yes**.

Step 7 Go back to **Device Manager > Network adapters**. **USB RNDIS6 Adapter** is displayed.

Figure 5-13 Normal display of the RNDIS driver



----End

5.11 Setting User Account Validity Period

Run the **chage** command to set the validity period of a user account for security purposes.

Command:

```
chage [-m mindays] [-M maxdays] [-d lastday] [-I inactive] [-E expiredate] [-W warndays] user
```

Table 5-4 describes the parameters.

Table 5-4 Parameters description

Parameter	Description
-m	Minimum time (in days) for which the password must be used. The password cannot be changed during this period. The value 0 indicates that the password can be changed at any time.
-M	Maximum validity period (days) of a password. The value 1 indicates that the validity check of the password can be disabled.

Parameter	Description
-d	Last password change date.
-l	Maximum idle period (in days) after which the user account will be disabled. After the specified time period has expired, the password will be invalid.
-E	Date when the user account expires. The user account is unavailable when the account validity period has expired.
-W	Number of days in advance users are notified that their passwords are about to expire.
-l	Lists the current settings. It helps non-privileged users to determine the time when their passwords or accounts expire.

 NOTE

- **Table 5-4** lists only common parameters. You can run the **chage --help** command to display detailed parameter description.
- The date is in the format of *YYYY-MM-DD*. For example, **chage -E 2020-12-01 test** indicates that the user account **test** will expire on December 1, 2020.
- **User** must be specified. Replace it with the actual user name. The default user name is **root**.

For example, to change the validity period of the **test** user to December 1, 2020, run the following command:

```
chage -E 2020-12-01 test
```

5.12 Installing CMake 3.5.2

1. Run the **wget** command to download the source code package of CMake to any directory on the server:

```
wget https://cmake.org/files/v3.5/cmake-3.5.2.tar.gz --no-check-certificate
```
2. Run the following command to go to the download directory and decompress the source code package:

```
tar -zxvf cmake-3.5.2.tar.gz
```
3. Go to the decompressed folder and run the following configuration, compilation, and installation commands:

```
cd cmake-3.5.2
./bootstrap --prefix=/usr
make
sudo make install
```
4. After the installation is complete, run the **cmake --version** command again to check the version number.

5.13 Configuring a System Network Proxy

The following procedure is a general method for configuring a network proxy. It may not be applicable to all network environments. The method of configuring the network proxy depends on the actual network environment.

Prerequisites

- Ensure that the network cable of the server is connected and the proxy server can connect to the external network.
- The configuration proxy is based on the condition that the server is located on an intranet and cannot be directly connected to the external network.

Configuring a System Network Proxy

Step 1 Log in to the user environment as the **root** user.

Step 2 Run the following command to edit the **/etc/profile** file:

```
vi /etc/profile
```

Add the following content to the file, save the file, and exit:

```
export http_proxy="http://user:password@proxyserverip:port"  
export https_proxy="http://user:password@proxyserverip:port"
```

In the preceding commands, **user** indicates the username on the intranet, **password** indicates the user password, **proxyserverip** indicates the IP address of the proxy server, and **port** indicates the port number.

Step 3 Run the following command to make the configuration take effect.

```
source /etc/profile
```

Step 4 Run the following command to check whether the external network is connected:

```
wget www.baidu.com
```

If the HTML file can be downloaded, the server is connected to the external network successfully.

NOTE

If a certificate error occurs when you use a proxy to connect to the network, you need to install the certificate of the proxy server before downloading third-party components.

----End

5.14 Parameters

One-click installation is supported in the command line. You can select parameters as required to complete the installation. All parameters are optional.

Installation command format: **./*.run** [options]

For details, see [Table 5-5](#).

NOTICE

If the parameters queried by running the `./*.run --help` command are not described in the following table, this parameter is reserved or applies to other chip versions. You do not need to pay attention to this parameter.

Table 5-5 Parameters supported by the installation package

Parameter	Description
<code>--help -h</code>	Queries help information.
<code>--version</code>	Queries version information.
<code>--info</code>	Queries software package construction information.
<code>--list</code>	Queries the software package list.
<code>--check</code>	Checks the consistency and integrity of software packages.
<code>--quiet</code>	Silent installation, skipping interactive messages.
<code>--noexec</code>	Decompresses a software package to the current directory without running the installation script. This parameter is used together with <code>--extract=<path></code> . The format is as follows: <code>--noexec --extract=<path></code>
<code>--extract=<path></code>	Decompresses a software package to a specified directory.
<code>--tar arg1 [arg2 ...]</code>	Runs the tar command on the software package. Use the arguments following tar as the command arguments. For example, the <code>--tar xvf</code> command indicates that the <code>.run</code> package will be decompressed to the current directory.
<code>--install</code>	Installs a software package. You can specify the installation path <code>--install-path=<path></code> or use the default installation path.
<code>--install-for-all</code>	Allows all users to have the same installation group permission. If this option is included in installation or upgrade command, all users have the same permission on the directories and files created by the runfile installer as the installation group. This parameter must be used together with one of the parameters <code>--install</code>, <code>--devel</code>, and <code>--upgrade</code>, for example, <code>./*.run --install --install-for-all</code>. NOTE Make sure the security risks are considered before you include this option.

Parameter	Description
--install-username=<user name>	Initial installation: You can specify the running user name. Otherwise, HwHiAiUser is used by default. Overwrite: The user name used in the last installation is adopted. This parameter must be used together with --install-usergroup=<usergroup>.
--install-usergroup=<user group>	Initial installation: You can specify the running user group name. Otherwise, HwHiAiUser is used by default. Overwrite: The user group name used in the last installation is adopted. This parameter must be used together with --install-username=<username>.
--install-path=<path>	Specifies the installation path. If you do not specify the installation path, the default installation path is used. - For installation as user root, the default installation path is /usr/local/Ascend. - For installation as a non-root user, the default installation path is \${HOME}/Ascend. If this parameter is used to specify the installation directory, the running user must have the read and write permissions on the specified installation directory.
--uninstall	Uninstalls the software that has been installed.
--upgrade	Upgrades the software that has been installed. The system automatically checks the version number. If the version number is not in ascending order, the upgrade cannot be performed.
--devel	Installs a software package in development mode, that is, install only the files required by the development environment.

6 FAQs

6.1 What Do I Do If the "Software Has Been Installed" During RUN Package Installation?

6.2 The error message "subprocess.CalledProcessError: Command '('lsb_release', '-a')' return non-zero exit status 1 " is displayed during pip3.7.5 installation.

6.3 What Do I Do If "Could not find a version that satisfies the requirement xxx" Is Displayed When pip3.7.5 install Is Run?

6.4 What Do I Do If a Redundant Mounted Disk Appears Due to Manual Removal of the SD Card During SD Card Creation?

6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?

6.6 What Do I Do If the Atlas 200 DK Cannot Connect to the Ubuntu Server?

6.1 What Do I Do If the "Software Has Been Installed" During RUN Package Installation?

Symptom

The following message is displayed during installation:

```
run package is already installed, install failed
```

Solution

The software package cannot be installed repeatedly. You need to uninstall the software package and then install it. For details, see [5.1.3 Uninstalling the Development Kit](#).

6.2 The error message "subprocess.CalledProcessError: Command '('lsb_release', '-a')' return non-zero exit status 1 " is displayed during pip3.7.5 installation.

Symptom

During dependency installation, the error message "subprocess.CalledProcessError: Command '('lsb_release', '-a')' return non-zero exit status 1 " is displayed when you run the pip3.7.5 install xxx command to install related software. The error message is as follows:

```
root@debian:/home/work_env/Python-3.7.5# pip3.7.5 install attrs
ERROR: Exception:
Traceback (most recent call last):
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_internal/cli/base_command.py", line 188, in main
    status = self.run(options, args)
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_internal/commands/install.py", line 286, in run
    with self._build_session(options) as session:
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_internal/cli/base_command.py", line 108, in _build_session
    index_urls=self.get_index_urls(options),
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_internal/download.py", line 559, in __init__
    self.headers["User-Agent"] = user_agent()
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_internal/download.py", line 144, in user_agent
    zip["name", "version", "id"], distro.linux_distribution())
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 122, in linux_distribution
    return distro.linux_distribution(full_distribution_name)
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 677, in linux_distribution
    self.version()
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 737, in version
    self.lsb_release_attr('release'),
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 899, in lsb_release_attr
    return self._lsb_release_info.get(attribute, '')
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 552, in __get__
    ret = obj._dict[self.fname] = self.f(obj)
  File "/usr/local/python3.7.5/lib/python3.7/site-packages/pip/_vendor/distro.py", line 1012, in _lsb_release_info
    stdout = subprocess.check_output(cmd, stderr=devnull)
  File "/usr/local/python3.7.5/lib/python3.7/subprocess.py", line 411, in check_output
    **kwargs).stdout
  File "/usr/local/python3.7.5/lib/python3.7/subprocess.py", line 512, in run
    output=stdout, stderr=stderr)
subprocess.CalledProcessError: Command '('lsb_release', '-a')' returned non-zero exit status 1.
```

Possible Causes

When the subprocess module of Python 3.7.5 is executed, the system displays a message indicating that the lsb_release.py module cannot be found when the lsb_release -a command is executed. The lib path of Python 3.7.5 is /usr/local/python3.7.5/lib/python3.7/. The lsb_release.py module does not exist in the path. Therefore, an error is reported.

Procedure

Step 1 Run the following command to search for the missing file **lsb_release.py**:

```
find / -name lsb_release
```

After the preceding command is executed, the following path is obtained. The path is only an example and may vary according to the actual situation.

```
/usr/bin/lsb_release
```

Step 2 Run the following command to delete the **/usr/bin/lsb_release** file in [Step 1](#):

```
rm /usr/bin/lsb_release
```

Step 3 Run the **pip3.7.5 list** command to check whether the fault is rectified.

----End

6.3 What Do I Do If "Could not find a version that satisfies the requirement xxx" Is Displayed When pip3.7.5 install Is Run?

Symptom

What Do I Do If "Could not find a version that satisfies the requirement xxx" Is Displayed When pip3.7.5 install Is Run?

Figure 6-1 Message displayed upon pip3 install

```
ascend@dgghisard32833:~$ pip3 install numpy --user
Collecting numpy
  Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError(<urllib3.connection.VerifiedHTTPSConnection object at 7f5f725b9320>: Failed to establish a new connection: [Errno 101] Network is unreachable.'): /simple/numpy/
  Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError(<urllib3.connection.VerifiedHTTPSConnection object at 7f5f725cd908>: Failed to establish a new connection: [Errno 101] Network is unreachable.'): /simple/numpy/
  Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError(<urllib3.connection.VerifiedHTTPSConnection object at 7f5f725cd908>: Failed to establish a new connection: [Errno 101] Network is unreachable.'): /simple/numpy/
  Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError(<urllib3.connection.VerifiedHTTPSConnection object at 7f5f725cd908>: Failed to establish a new connection: [Errno 101] Network is unreachable.'): /simple/numpy/
  Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NewConnectionError(<urllib3.connection.VerifiedHTTPSConnection object at 7f5f725cd908>: Failed to establish a new connection: [Errno 101] Network is unreachable.'): /simple/numpy/
Could not find a version that satisfies the requirement numpy (from versions: )
```

Possible Causes

The pip source is not configured.

Procedure

Configure the pip source as follows:

Step 1 Run the following command as the installation user of the software package:

```
cd ~/.pip
```

If a message indicating that the directory does not exist is displayed, run the following command to create the directory:

```
mkdir ~/.pip
cd ~/.pip
```

Create a **pip.conf** file in the **.pip** directory:

```
touch pip.conf
```

Step 2 Edit the **pip.conf** file.

Run the **vi pip.conf** command to open the **pip.conf** file and edit the file as follows.

```
[install]
#Configure the trusted host as required.
trusted-host=cmc-cd-mirror.rnd.huawei.com
[global]
#Configure the sources as required.
index-url=http://cmc-cd-mirror.rnd.huawei.com/pypi/simple/
```

Step 3 Run the **:wq!** command to save the file and exit.

----End

6.4 What Do I Do If a Redundant Mounted Disk Appears Due to Manual Removal of the SD Card During SD Card Creation?

If the SD card is manually removed during SD card creation a redundant temporarily mounted disk is generated. You can perform the following steps to remove it.

Step 1 Log in to the Ubuntu server as a common user and run the **su - root** command to switch to the **root** user.

Step 2 Run the **df -h** command to view the temporarily mounted **/dev/loop0** disk.

```
root@kickseed:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0      745M  745M   0   100% /home/ubuntu/studio/scripts/180919002200
/dev/sdc1       118G   60M  112G   1%  /home/ubuntu/studio/scripts/sd_mount_dir
```

Step 3 Run the **umount** command to unmount the disk. Replace **/dev/loop0** and **/dev/sdc1** in the commands based on the actual query result in [Step 2](#).

```
root@kickseed:~# umount /dev/loop0
root@kickseed:~# umount /dev/sdc1
```

If "target is busy" is displayed, restart the Ubuntu server and repeat [Step 1](#) to [Step 3](#).

----End

6.5 What Do I Do If the Trust Relationship Between the Ubuntu Server and the Developer Board Fails to Be Established?

Symptom

On the Ubuntu server, the following command is run to connect to the Atlas 200 DK developer board in SSH mode. A message is displayed, indicating that no trust relationship exists.

The following command is run on the Ubuntu server to re-establish the trust relationship:

```
ssh-keygen -f "$HOME/.ssh/known_hosts" -R 192.168.1.2
```

192.168.1.2 is the IP address of the Atlas 200 DK developer board.

The following error is reported:

```
ECDSA host key for 192.168.1.2 has changed and you have requested strict checking.
```

Solution

This error is caused by the invalid SSH information stored on the local host. Therefore, you need to clear the local SSH information and establish the connection again.

Step 1 On the Ubuntu server, clear the public key information about the connection to the **192.168.1.2** host of the current user.

```
ssh-keygen -R 192.168.1.2
```

Step 2 Re-connect to the Atlas 200 DK developer board in SSH mode.

```
ssh HwHiAiUser@192.168.1.2
```

When the following information is displayed, enter **yes** to re-establish the SSH connection.

```
The authenticity of host '192.168.1.2' can't be established.  
ECDSA key fingerprint is 53:b9:f9:30:67:ec:34:88:e8:bc:2a:a4:6f:3e:97:95.  
Are you sure you want to continue connecting (yes/no)?
```

----End

6.6 What Do I Do If the Atlas 200 DK Cannot Connect to the Ubuntu Server?

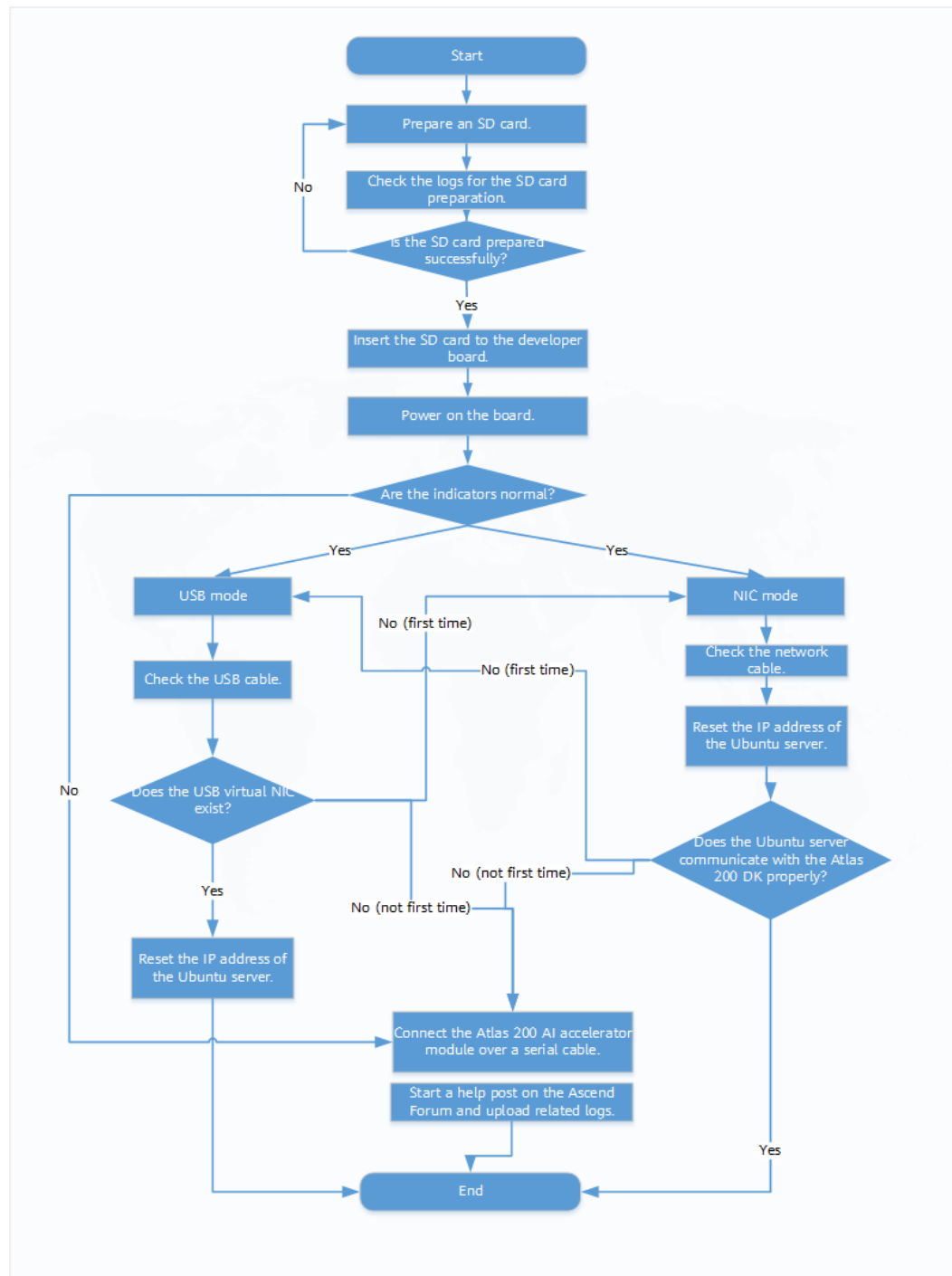
Symptom

The symptoms are as follows:

- After the Atlas 200 DK is powered on with the prepared SD card inserted, the states of LED1 and LED2 indicators are abnormal.
- After the Atlas 200 DK is powered on and started with the prepared SD card inserted and the Ubuntu server connected in USB mode, no virtual NIC is identified on the Ubuntu server.
- After the Atlas 200 DK is powered on and started with the prepared SD card inserted, the Ubuntu server connected in NIC mode, and Ubuntu server NIC configured, the Ubuntu server fails to communicate with the Atlas 200 DK.

Fault Locating

Perform troubleshooting by referring to [Figure 6-2](#).

Figure 6-2 Troubleshooting on Atlas 200 DK connection failure

Solution

Step 1 Ensure that the SD card is made correctly and successfully.

In the **sd_card_making_log** file in the directory where the card preparation script is located, check whether the card is made successfully. If not, try again by referring to [4.1.1 Creating an SD Card](#).

Step 2 Insert the SD card into the Atlas 200 DK and power on the board.

- If LED1 and LED2 on the Atlas 200 DK are normal, that is, LDE1 and LDE2 are both on after the Atlas 200 DK is started, go to [Step 3](#).
- If LED1 and LED2 on the Atlas 200 DK are abnormal, that is, LED1 and LED2 are not on at the same time 15 minutes after the Atlas 200 DK is started, view the system software installation logs and Atlas 200 DK startup logs by referring to [Exception Handling](#).
If the fault persists, go to [Step 4](#).

Step 3 Connect the Atlas 200 DK to the Ubuntu server.

- If the Ubuntu server is connected to the Atlas 200 DK in USB mode, but the virtual USB NIC is not displayed.
Check the USB network cable and ensure that both ends of the USB network cable are properly connected.
If the USB virtual NIC is still not identified on the Ubuntu server, connect the Ubuntu server to the Atlas 200 DK in NIC mode.
- If the Ubuntu server is connected to the Atlas 200 DK in NIC mode, but the Ubuntu server fails to communicate with the Atlas 200 DK after the IP address is configured.
Check the network cable and ensure that both ends of the network cable are properly connected. Then, reconfigure the IP address of the NIC on the Ubuntu server.
If the Ubuntu server still fails to communicate with the Atlas 200 DK, connect the Ubuntu server to the Atlas 200 DK in USB mode.

If the Ubuntu server fails to connect to the Atlas 200 DK in either USB or NIC mode, go to [Step 4](#).

Step 4 Connect the AI accelerator module of the Atlas 200 DK to the Ubuntu server over a serial cable by referring to [5.4 Connecting the Atlas 200 DK over a Serial Port](#).

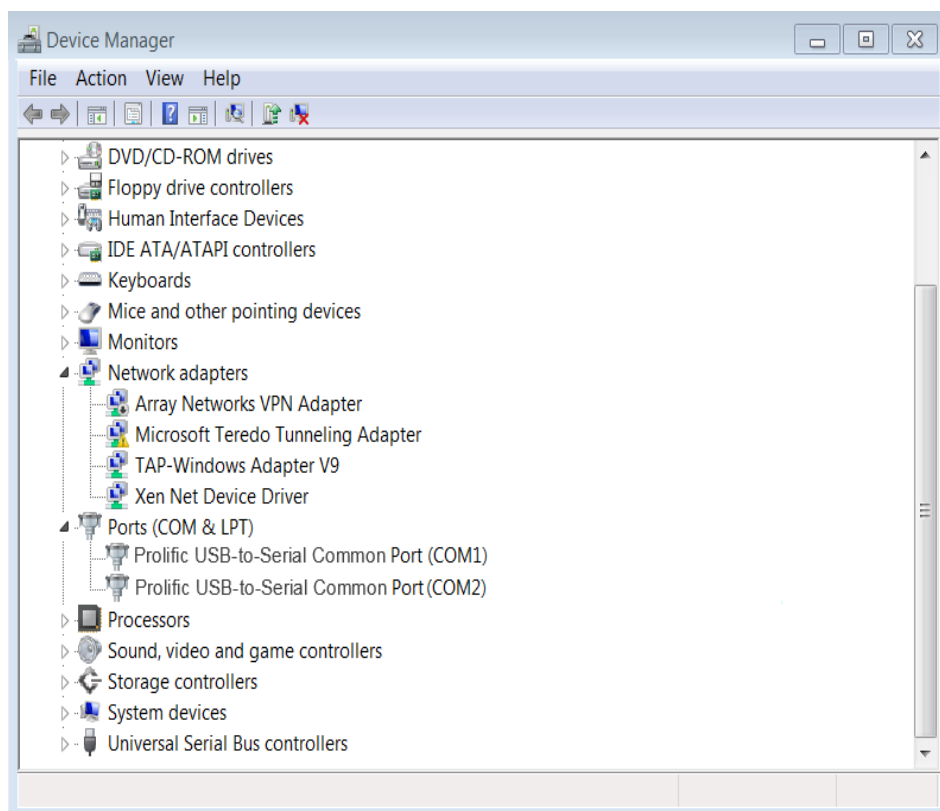
Step 5 Install the network debugging tool and USB-to-serial driver on the Ubuntu server.

- Recommended network debugging tool: IPOP
- USB-to-serial driver: PL2303 driver

Step 6 Start the network project debugging tool, for example, IPOP. The serial port window is displayed.

1. Click the **Terminal** tab page.
2. On the menu bar, click . The **Connect List** dialog box is displayed.
3. Configure the connection.
 - **ConnName**: indicates a user-defined connection name.
 - **Type**: indicates a port type. Choose **COMX**. You can view the available COM ports in the device manager of the computer. Remove and insert the serial cable on the Ubuntu server to determine the COM port used by the Atlas 200 DK, as shown in [Figure 6-3](#).

Figure 6-3 Viewing COM ports



- Set the baud rate to **115200**.

4. Click **OK**.

Step 7 Power on the Atlas 200 DK, and view the Atlas 200 DK boot information in the COM connection window of the IPOPOP tool.

There are many startup logs. Click  on the menu bar to save the startup logs to the installation directory of the IPOPOP tool. When this button changes to , a message is displayed at the bottom of the IPOPOP tool, indicating that the log file is saved. You can obtain the log file named after the current time from the installation directory of the IPOPOP tool according to the message.

Step 8 Start a help post on the [Ascend Forum](#) and upload the startup log file as an attachment. Huawei engineers will provide technical support for you.

----End